北京智源人工智能研究院
BEIJING ACADEMY OF ARTIFICIAL INTELLIGENCE

GPU架构与编程介绍

智源人工智能研究院

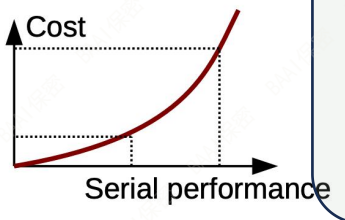
2025.8.17

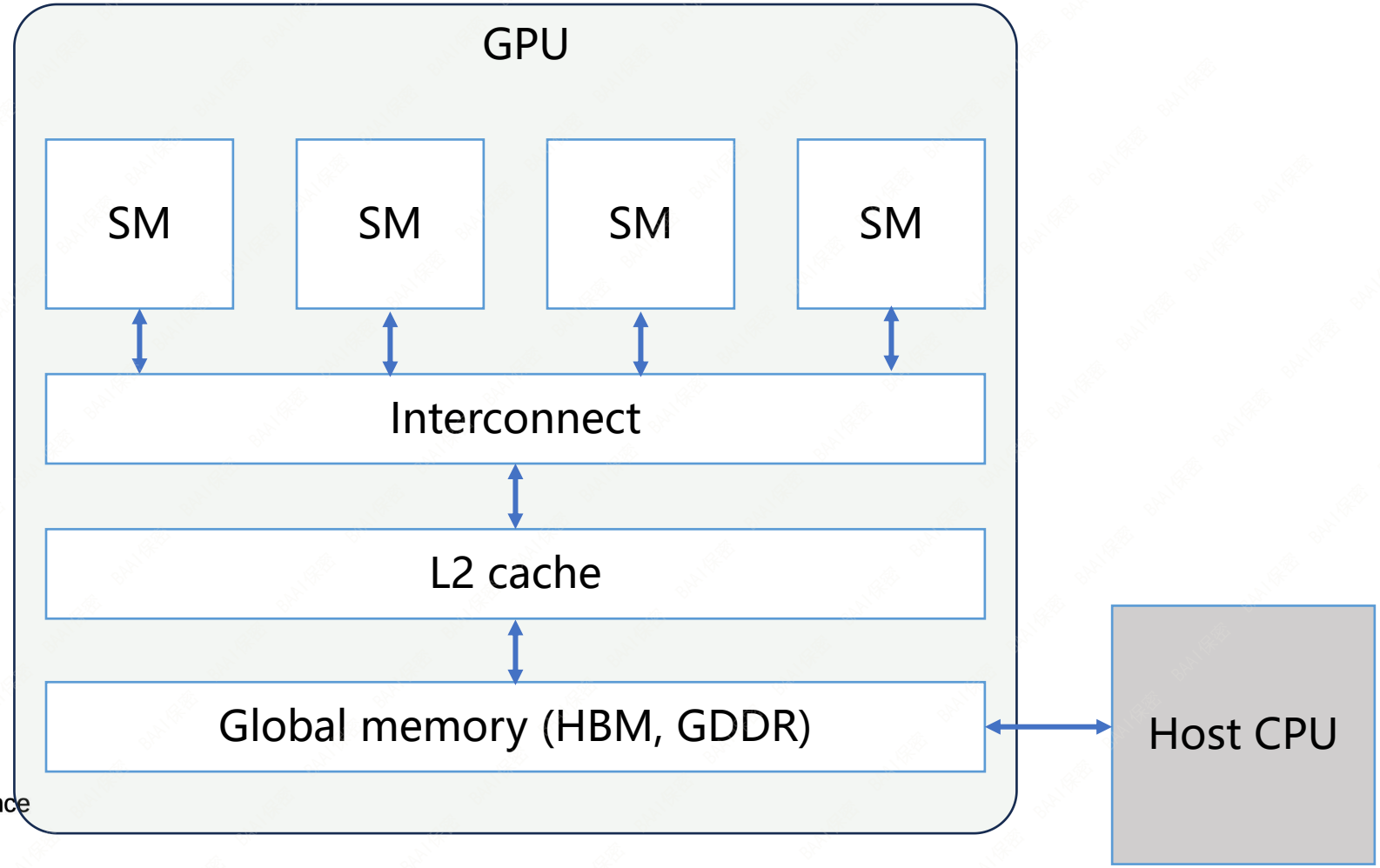
内容

- GPU 架构
- CUDA vs Triton
- 延时与带宽
- Tensorcore

GPU回顾

- 早期图形加速卡
  - 256 1999
- GPGPU
  - Tesla，2006
  - Stream processor
- CUDA
  - 大规模并行处理
  - Top500占比
    - 2010: 2%
    - 2018: 22%
- 通用高性能CPU发展受阻
  - 内存墙
  - 功耗墙
  - ILP受限
  -
- 广泛应用：
  - 游戏产业，科学与工程模拟，人工智能，医学影像，金融





GPU系统架构

- 计算层次
  - GPU -> SM -> CUDA cores, Tensor cores
- GPUs
  - 互联
    - PCIe：以A100为例，PCIe v4 x 16，64 GB/s 双向带宽
    - SXM，NVLink直连主板，增强server端GPU间互联速度
    - NVLink：
      - A100, 12 x NVLink sublink 3.0 = 12 x 25G/bs x 2 = 600 GB/s 双向带宽
      - H100, 18 x NVLink sublink 4.0, 900 GB/s

HGX A100

A100 PCIe v4

GPU系统架构

- 计算层次
  - GPU -> SM -> CUDA cores, Tensor cores
- GPUs
  - SMs
    - V100: 80 SMs
    - A100: 108 SMs
    - H100: 132 SMs
  - Shared L2 cache
    - V100: 6,144 KB (~77 KB/SM)
    - A100: 40 MB (~379 KB/SM)
    - H100: 60 MB (~465 KB/SM)
  - Shared device memory
    - V100: HBM2 40 GB (~512 MB/SM)
    - A100: HBM2e 80 GB (~758 MB/SM)
    - H100: HBM3 96 GB (~745 MB/SM)
  - Device memory bandwidth
    - A100: 1555 GB/s
    - H100: 3T GB/s

Figure 6. GA100 Full GPU with 128 SMs (A100 Tensor Core GPU has 108 SMs)

GPU系统架构

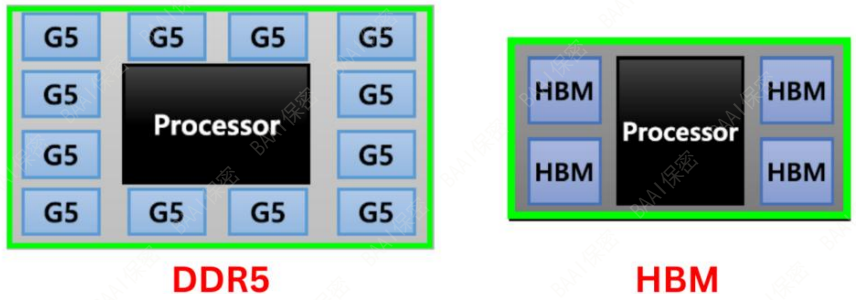
HBM IO 接口宽度 = 1024 bits

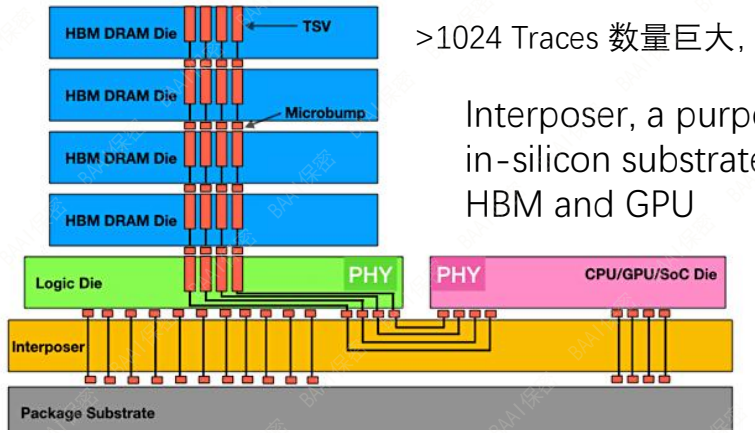
- 计算层次
  - GPU -> SM -> CUDA cores, Tensor cores
- GPUs
  - HBM，高带宽，低延时，低容量，高造价
    - 使用 3d 堆叠构造存储芯片，空间更紧凑，相反，DDR 是2d结构，只能平铺扩容
    - HBM IO 接口更宽(1024bits)，利于并行数据移动，带宽超过 DDR 数量级以上，DDR5 33.6 GB/s，HBM2e 410 GB/s
    - 3d 结构缩短连接距离(TSV垂直互联)，HBM相对DDR具有功耗优势
    - HBM 趋势, 信号传输速率从 2Gbps 到 6.4 Gbps

Bandwidth = IO speed x **IO bits / 8**

Sources: SK hynix and Rambus

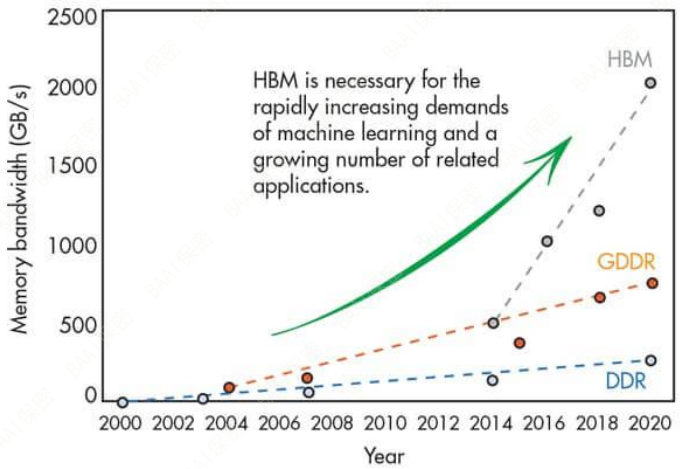
HBM vs. DDR





>1024 Traces 数量巨大，PCB 无法支持

Interposer, a purposely built in-silicon substrate connecting HBM and GPU



GPU系统架构

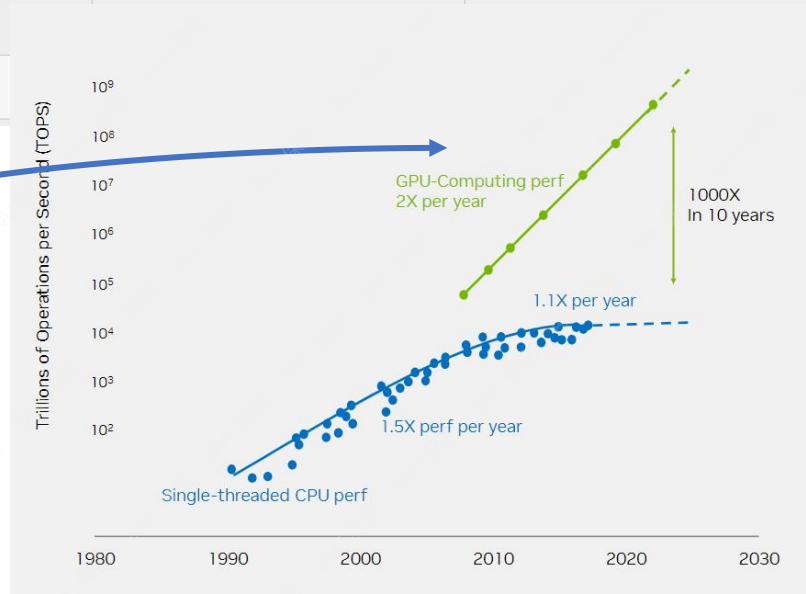
- 计算层次
  - GPU -> SM -> CUDA cores, Tensor cores
- SM ( )
  - 4 SPs, 单个SP粗略对应单个CPU core
  - SM与CPU core的区别
    - GPU SMSP 数量更多， 4 x 100+
    - CPU core 特征是 深流水 superscalar，乱序执行，复杂分支预测，少量寄存器，大缓存
    - GPU core 特性是指令顺序执行+SIMD 或 SIMT，大量寄存器，缓存占用die空间相对较小
    - GPU 访存能力更强，自动访存合并，banked shared memory，原子操作支持global 和 shared memory
    - GPU 指令集更复杂，指令集兼容性差
    - 总体而言，GPU 以提升吞吐为首要目标，可支持成千上万个线程同时执行
    - 相反，CPU core设计哲学以缩短单线程执行时间为圭臬，因此大量die空间用于避免stall，包括缓存、乱序执行、分支预测、投机执行等

A100 SM

GPU系统架构

- 计算层次
  - GPU -> SM -> CUDA cores, Tensor cores
- CUDA cores
  - 通用计算单元，执行FP32/FP64浮点和整数运算
  - 64 fp32 cuda cores per SM
- Tensor cores
  - 矩阵乘计算单元，支持混合精度计算(FP16, TF32, INT8/INT4等)
  - A100 第三代 tensor core 每时钟周期可执行 8 x 4 x 8 (=256) 个 FP16 FMA 计算
  - 4 tensor cores / SM, 4 x 256 = 1024 FP16 FMA, = 2048 FLOPs per clock
  - 贡献一半以上AI算力 FLOPs
  - 编程噩梦 😵

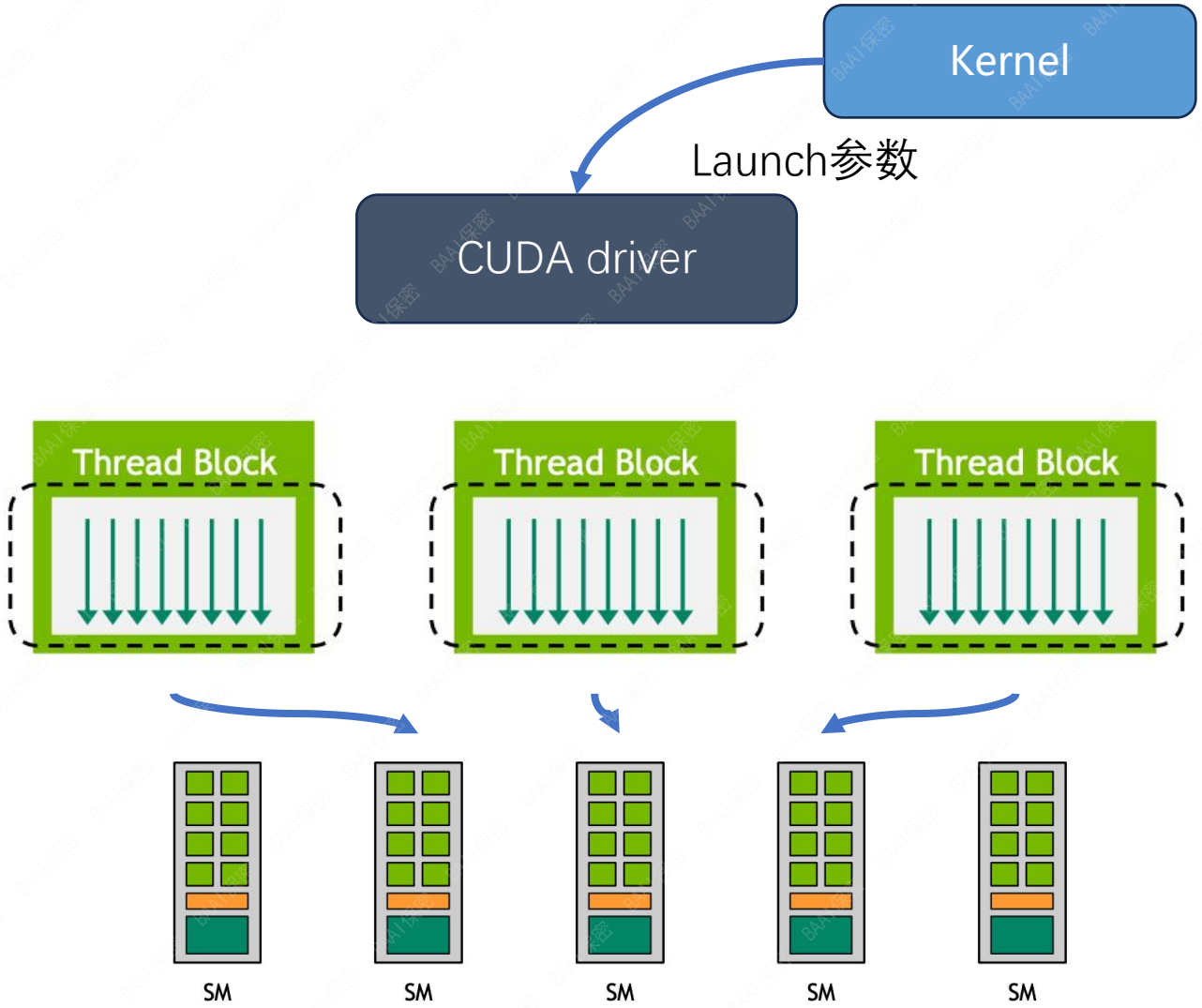
Form Factor	H100 SXM	H100 PCIe
FP64	34 teraFLOPS	26 teraFLOPS
FP64 Tensor Core	67 teraFLOPS	51 teraFLOPS
FP32	67 teraFLOPS	51 teraFLOPS
TF32 Tensor Core	989 teraFLOPS*	756teraFLOPS*
BFLOAT16 Tensor Core	1,979 teraFLOPS*	1,513 teraFLOPS*
FP16 Tensor Core	1,979 teraFLOPS*	1,513 teraFLOPS*
FP8 Tensor Core	3,958 teraFLOPS*	3,026 teraFLOPS*
INT8 Tensor Core	3,958 TOPS*	3,026 TOPS*
GPU memory		
GPU memory bandwidth		



Huagn's law

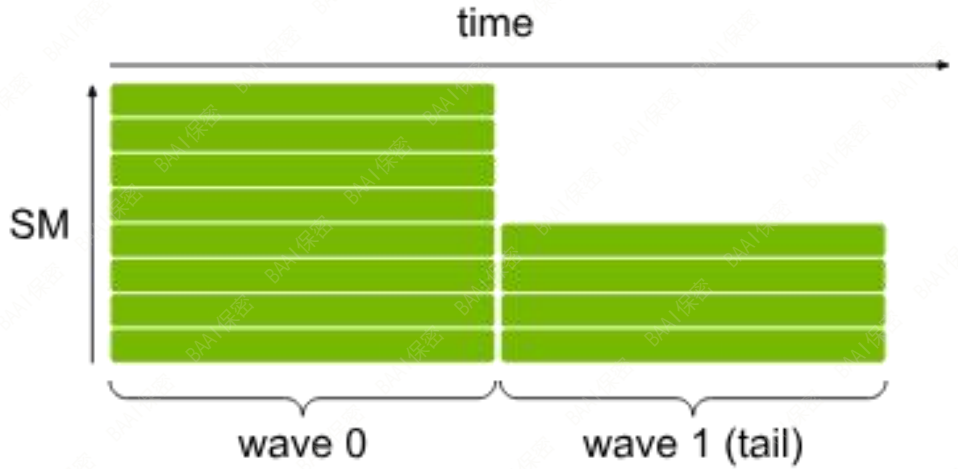
GPU线程执行模型

- 同时执行大量线程
- 线程分层结构
  - 独立线程块
  - 硬件调度线程块到SM上执行
- 流式处理
  - 线程块数多于SM，线程数多于物理核心数，GPU能自动切换线程执行，隐藏指令延迟
  - 利用强大的硬件调度能力，通常任务切成小块的性能表现更好
- SM内部提供共享内存，支持线程块内进行快速通信和同步



GPU线程执行模型

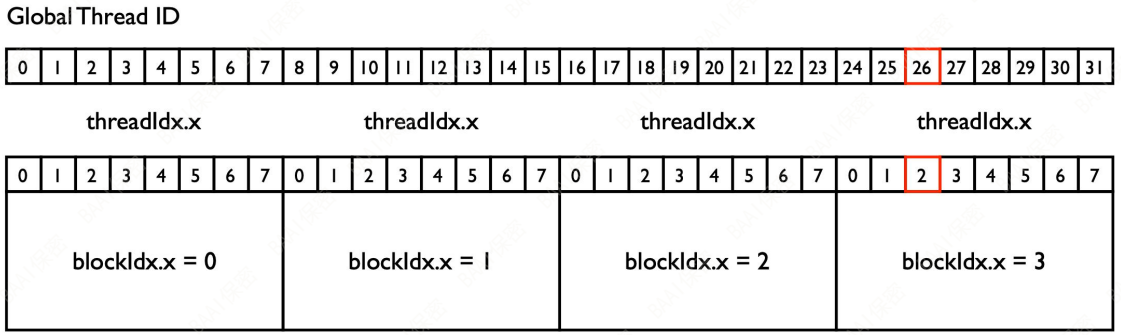
为获得负载均衡，通常情况线程块数量达到SM数倍以上
反之则容易出现尾部wave空洞效应

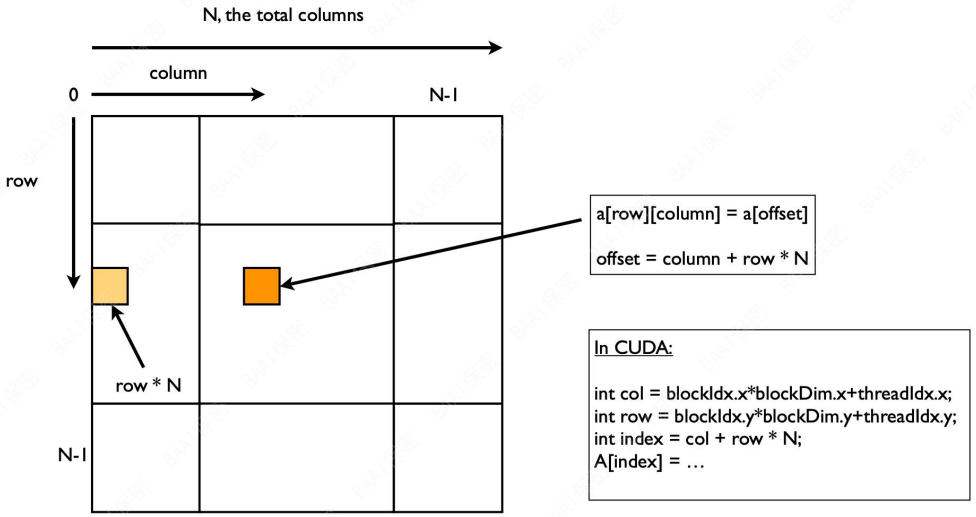


GPU 线程组织模式

- Grid：一次 Kernel launch 的线程集合
  - 3维线程块空间
  - 线程块相互独立
  - blockIdx.x, blockIdx.y, blockIdx.z，CUDA 中线程块在grid中唯一的3维坐标
  - Kernel 运行时参数，调用时设定
- ThreadBlock
  - 3维线程空间
  - blockDim.x, blockDim.y, blockDim.z ，3维线程块大小
  - 同样也是 Kernel 运行时参数
- Warp（AMD Wavefront）
  - 固定数量的线程组，nv 32，amd 64
  - 硬件调度单元
- Thread
  - 最小运行时状态维护单位
  - threadIdx.x, threadIdx.y, threadIdx.z，线程在线程块中的唯一3维坐标

1d grid, 1d thread block
Grid size = (4, , ,)
Block size = (8, , ,)





高维 grid 和线程块组织通常用于映射高维数据

GPU 编程：CUDA 与 Triton

- CUDA
  - 以线程为中心的编程模式，kernel代码描述单线程执行逻辑，标量操作语义
  - 手动排布线程块与数据的映射关系
  - 手动控制线程块级别的同步
  - 手动分配线程块共享内存
- Triton
  - 以数据块为中心的编程模式，kernel代码描述整个线程块的数据并行执行逻辑数据并行操作语义
  - 编译器自动规划数据映射到线程块的layout
  - 无需手动插入线程块同步操作
  - 无法直接分配和操作线程块共享内存
- 整体编程体验
  - Triton相对更易编程，但另一方面限制了操控自由度，且对新硬件特性的支持不到位
  - CUDA提供更多发挥余地，但初学者容易不得要领，性能反而不如Triton
  - Triton 与 CUDA 差异一半体现软件工程上，另一半体现在性能表现上

Triton 与 CUDA 的软件工程角度对比

- CUDA 与 C++ 软件框架实现无缝对接，对模版元编程和复杂数据结构的编译支持到位，支持灵活的代码复用，适合构建大型复杂软件框架。

```
template <bool is_scatter_like, typename scalar_t, typename index_t>
struct _cuda_scatter_gather_internal_kernel {
  template <typename func_t>
  void operator() (
    TensorIterator& iter,
    int64_t index_size,
    int64_t index_stride,
    int64_t numel,
    const func_t& f
  ) {
    ...
    auto offset_calc = make_offset_calculator<3>(iter);
    auto loop = [=]C10_DEVICE(int i) {
      auto offsets = offset_calc.get(i);
      ...
      int64_t idx_dim = *(index_t*)(index_ptr + offsets[2]);
      f(
        (scalar_t*)(self_ptr + offsets[0]),
        is_scatter_like ? idx_dim * index_stride : 0,
        numel,
        (scalar_t*)(src_ptr + offsets[1]) + (is_scatter_like ? 0 : idx_dim * index_stride)
      );
    };

    _launch_scatter_gather_kernel<num_threads(), thread_work_size()>(iter.numel(), loop);

  }
}; // struct _cuda_scatter_gather_internal_kernel
```

Pytorch gather kernel 的元编程实现

Triton 与 CUDA 的软件工程角度对比

北京智源人工智能研究院
BEIJING ACADEMY OF ARTIFICIAL INTELLIGENCE

- Triton 则适合直接在 Python+Pytorch 环境中嵌入式定义自定义算子，但用其实现通用算子则颇为蹩脚

```
class GatherFunction:
    def __init__(self):
        self.pid = os.getpid()
        self.overloads: Mapping[str, Callable] = {}

    def __call__(self, *args, **kwargs):
        key = f"{self.arg_key(*args)}"
        if key in self.overloads:
            overload = self.overloads[key]
        else:
            code = IndentedBuffer()
            code = generate_code(
                args,
                "_gather_wrapper",
                "_gather_flaggems_jit_function",
                code,
            )

            file_name = f"gather_rank_{key}.py"
            file_path = code_cache_dir() / file_name
            write_atomic(file_path, code.getvalue())

            # load
            spec = importlib.util.spec_from_file_location(
                f"_gen_module_rank_{key}",
                file_path,
            )

            m = importlib.util.module_from_spec(spec)
            spec.loader.exec_module(m)
            overload = getattr(m, "_gather_wrapper")
            self.overloads[key] = overload

        return overload(*args, **kwargs)

    def arg_key(self, *args):
        return args[0].ndim
```

```
def generate_code(
    inputs: Tuple[Any],
    wrapper_name: str,
    kernel_name: str,
    code: IndentedBuffer,
) -> IndentedBuffer:
    rank = inputs[0].ndim

    code = generate_imports(code)
    code = generate_gather_kernel(rank, kernel_name, code)
    code = generate_gather_wrapper(rank, wrapper_name, kernel_name, code)
    return code


def generate_gather_wrapper(
    rank: int,
    wrapper_name: str,
    kernel_name: str,
    code: IndentedBuffer,
) -> IndentedBuffer:
    code.writeline(f"def {wrapper_name}(inp, dim, index, out, dim_stride, N):")
    with code.indent():
        code.writeline("inp_shape = inp.shape")
        code.writeline("inp_stride = inp.stride()")
        code.writeline("index_shape = index.shape")
        code.writeline("index_stride = index.stride()")
        code.writeline("out_shape = out.shape")
        code.writeline("out_stride = out.stride()")
        code.writeline("grid = lambda meta: (triton.cdiv(N, meta['BLOCK_SIZE_N']), )")
        code.writeline(f"{kernel_name}[grid](")
        with code.indent():
            args = [
                "inp, ",
                "index, ",
                "out, ",
            ]
            args += [f"inp_shape[{i}], " for i in range(rank)]
            args += [f"index_shape[{i}], " for i in range(rank)]
            args += [f"out_shape[{i}], " for i in range(rank)]
            args += [f"inp_stride[{i}], " for i in range(rank)]
            args += [f"index_stride[{i}], " for i in range(rank)]
            args += [f"out_stride[{i}], " for i in range(rank)]
            args += [
```

FlagGems gather kernel 的代码生成实现

```
def generate_gather_kernel(
    rank: int,
    kernel_name: str,
    code: IndentedBuffer,
) -> IndentedBuffer:
    # make the inlined function visible in the context
    code.newline()

    code.writeline("@libentry()")
    code.writeline("@triton.heuristics({'BLOCK_SIZE_N': lambda args: 512})")
    code.writeline("@triton.jit")
    code.writeline(f"def {kernel_name}(")
    with code.indent():
        args = [
            "inp, ",
            "index, ",
            "out, ",
        ]
        args += [f"inp_shape{i}," for i in range(rank)]
        args += [f"index_shape{i}, " for i in range(rank)]
        args += [f"out_shape{i}, " for i in range(rank)]
        args += [f"inp_stride{i}, " for i in range(rank)]
        args += [f"index_stride{i}, " for i in range(rank)]
        args += [f"out_stride{i}, " for i in range(rank)]
        args += ["dim, ", "dim_stride, ", "N, ", "BLOCK_SIZE_N: tl.constexpr, "]
        code.writelines(args)
    code.writeline("):")

    with code.indent():
        code.writeline("pid = tle.program_id(0)")
        code.writeline(
            "offset = pid * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N).to(tl.int64)"
        )
        code.newline()
        code.writeline("cur_offset = offset")
        for i in range(rank - 1, -1, -1):
            code.writeline(f"index_idx{i} = cur_offset % index_shape{i}")
            code.writeline(f"cur_offset = cur_offset // index_shape{i}")
        code.newline()
        comp = [f"index_idx{i} * index_stride{i}" for i in range(rank)]
        code.writeline(f"index_offset = {' + '.join(comp)}")
        code.writeline("mask = offset < N")
        code.writeline("cur_index = tl.load(index + index_offset, mask=mask, other=0)")
        code.newline()
```

BAAI

Triton 与 CUDA 的软件工程角度对比

- CUDA kernel 编程更倾向于静态调优，为复杂计算密集型算子提供大量预编译kernel，使用grid stride loop 编程范式增强灵活性

```cpp
__global__ void elementwise_sumOfSquares(float *a, float *b, float *c, int N) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;

    int gsl_stride = blockDim.x * gridDim.x;
    #pragma unroll
    for (; index < N; index += gsl_stride) {
        c[index] = a[index] * a[index] + b[index] * b[index];
    }
}
```

线程块及Tile大小则凭经验优化后固定

示例：逐点计算向量的平方和

```
Time taken: 7.497408 ms, gbps: 1600.55, N = 1073741824, num_warps = 2
Time taken: 5.543072 ms, gbps: 2164.86, N = 1073741824, num_warps = 4
Time taken: 5.538976 ms, gbps: 2166.47, N = 1073741824, num_warps = 8
Time taken: 4.774720 ms, gbps: 2513.24, N = 1073741824, num_warps = 16
Time taken: 4.668096 ms, gbps: 2570.64, N = 1073741824, num_warps = 32
```

固定grid size = 1320
最优 num warps 配置

```
Time taken: 4.545440 ms, gbps: 2640.01, N = 1073741824, gridSize = 1320, num_warps = 32
Time taken: 4.910720 ms, gbps: 2443.63, N = 1073741824, gridSize = 670, num_warps = 32
Time taken: 4.747520 ms, gbps: 2527.64, N = 1073741824, gridSize = 260, num_warps = 32
Time taken: 5.988640 ms, gbps: 2003.79, N = 1073741824, gridSize = 132, num_warps = 32
Time taken: 4.907552 ms, gbps: 2445.21, N = 1073741824, gridSize = 1320, num_warps = 16
Time taken: 5.477504 ms, gbps: 2190.78, N = 1073741824, gridSize = 670, num_warps = 16
Time taken: 6.001824 ms, gbps: 1999.39, N = 1073741824, gridSize = 260, num_warps = 16
Time taken: 9.007776 ms, gbps: 1332.18, N = 1073741824, gridSize = 132, num_warps = 16
```

Grid size 减少到接近 SM 数量时性能明显下滑，GPU架构更适合 over subscription

Triton 与 CUDA 编程习惯的差异

- Triton 借助JIT编译和autotuning接口，简单情况下可免于关注数据tile线程调度细节

```python
@triton.autotune(
    configs=[
        triton.Config({'BLOCK_SIZE': b}, num_warps = w)
        for b, w
        in itertools.product((128, 256, 512, 1024), (2, 4, 8, 16))
    ],
    key = ['N']
)
@triton.jit(do_not_specialize=['N'])
def elementwise_sumOfSquares(x_ptr, y_ptr, out_ptr, N, BLOCK_SIZE: tl.constexpr):
    pid  = tl.program_id(0)
    offset = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)
    mask = offset < N
    x = tl.load(x_ptr + offset, mask=mask)
    y = tl.load(y_ptr + offset, mask=mask)
    z = x * x + y * y
    tl.store(out_ptr + offset, z, mask=mask)
```

```
Time taken: 4.125200 ms, gbps: 2908.9498456943084, N = 1073741824, BLOCK_SIZE = 1024, num_warps = 16
```

简单的调优就轻而易举战胜手动调试过的 Cuda kernel

Triton 隐藏了什么


```
#blocked = #ttg.blocked<{sizePerThread = [2], threadsPerWarp = [32], warpsPerCTA = [16], order = [0]}>
#loc = loc("/home/tongxin/bandwidth.py":33:0)
module attributes {"ttg.num-ctas" = 1 : i32, "ttg.num-warps" = 16 : i32, ttg.target = "cuda:90", "ttg.thread
  tt.func public @elementwise_sumOfSquares(%arg0: !tt.ptr<f32> {tt.divisibility = 16 : i32} loc("/home/tongx
    %c1024_i32 = arith.constant 1024 : i32 loc(#loc1)
    %0 = tt.get_program_id x : i32 loc(#loc2)
    %1 = arith.muli %0, %c1024_i32 : i32 loc(#loc3)
    %2 = tt.make_range {end = 1024 : i32, start = 0 : i32} : tensor<1024xi32, #blocked> loc(#loc4)
    %3 = tt.splat %1 : i32 -> tensor<1024xi32, #blocked> loc(#loc5)
    %4 = arith.addi %3, %2 : tensor<1024xi32, #blocked> loc(#loc5)
    %5 = tt.splat %arg3 : i32 -> tensor<1024xi32, #blocked> loc(#loc6)
    %6 = arith.cmpi slt, %4, %5 : tensor<1024xi32, #blocked> loc(#loc6)
    %7 = tt.splat %arg0 : !tt.ptr<f32> -> tensor<1024x!tt.ptr<f32>, #blocked> loc(#loc7)
    %8 = tt.addptr %7, %4 : tensor<1024x!tt.ptr<f32>, #blocked>, tensor<1024xi32, #blocked> loc(#loc7)
    %9 = tt.load %8, %6 : tensor<1024x!tt.ptr<f32>, #blocked> loc(#loc8)
    %10 = tt.splat %arg1 : !tt.ptr<f32> -> tensor<1024x!tt.ptr<f32>, #blocked> loc(#loc9)
    %11 = tt.addptr %10, %4 : tensor<1024x!tt.ptr<f32>, #blocked>, tensor<1024xi32, #blocked> loc(#loc9)
    %12 = tt.load %11, %6 : tensor<1024x!tt.ptr<f32>, #blocked> loc(#loc10)
    %13 = arith.mulf %9, %9 : tensor<1024xf32, #blocked> loc(#loc11)
    %14 = arith.mulf %12, %12 : tensor<1024xf32, #blocked> loc(#loc12)
    %15 = arith.addf %13, %14 : tensor<1024xf32, #blocked> loc(#loc13)
    %16 = tt.splat %arg2 : !tt.ptr<f32> -> tensor<1024x!tt.ptr<f32>, #blocked> loc(#loc14)
    %17 = tt.addptr %16, %4 : tensor<1024x!tt.ptr<f32>, #blocked>, tensor<1024xi32, #blocked> loc(#loc14)
    tt.store %17, %15, %6 : tensor<1024x!tt.ptr<f32>, #blocked> loc(#loc15)
    tt.return loc(#loc16)
  } loc(#loc)
} loc(#loc)
```

- In-flight bytes 增加，提高了有效吞吐
- 然而随着 block size 的增加，Triton 会不会简单增大 sizePerThread？这会导致global memory 访问的stride过大，无法 coalesce

Triton 隐藏了什么


```
Time taken: 4.145984 ms, gbps: 2894.367054912045, N = 1073741824, BLOCK_SIZE = 16384, num_warps = 16
```

```
#blocked = #ttg.blocked<{sizePerThread = [4], threadsPerWarp = [32], warpsPerCTA = [16], order = [0]}>
#loc = loc("/home/tongxin/bandwidth.py":33:0)
module attributes {"ttg.num-ctas" = 1 : i32, "ttg.num-warps" = 16 : i32, ttg.target = "cuda:90", "ttg.thread
  tt.func public @elementwise_sumOfSquares(%arg0: !tt.ptr<f32> {tt.divisibility = 16 : i32} loc("/home/tongx
    %c16384_i32 = arith.constant 16384 : i32 loc(#loc1)
    %0 = tt.get_program_id x : i32 loc(#loc2)
    %1 = arith.muli %0, %c16384_i32 : i32 loc(#loc3)
    %2 = tt.make_range {end = 16384 : i32, start = 0 : i32} : tensor<16384xi32, #blocked> loc(#loc4)
    %3 = tt.splat %1 : i32 -> tensor<16384xi32, #blocked> loc(#loc5)
    %4 = arith.addi %3, %2 : tensor<16384xi32, #blocked> loc(#loc5)
    %5 = tt.splat %arg3 : i32 -> tensor<16384xi32, #blocked> loc(#loc6)
    %6 = arith.cmpi slt, %4, %5 : tensor<16384xi32, #blocked> loc(#loc6)
    %7 = tt.splat %arg0 : !tt.ptr<f32> -> tensor<16384x!tt.ptr<f32>, #blocked> loc(#loc7)
    %8 = tt.addptr %7, %4 : tensor<16384x!tt.ptr<f32>, #blocked>, tensor<16384xi32, #blocked> loc(#loc7)
    %9 = tt.load %8, %6 : tensor<16384x!tt.ptr<f32>, #blocked> loc(#loc8)
    %10 = tt.splat %arg1 : !tt.ptr<f32> -> tensor<16384x!tt.ptr<f32>, #blocked> loc(#loc9)
    %11 = tt.addptr %10, %4 : tensor<16384x!tt.ptr<f32>, #blocked>, tensor<16384xi32, #blocked> loc(#loc9)
    %12 = tt.load %11, %6 : tensor<16384x!tt.ptr<f32>, #blocked> loc(#loc10)
    %13 = arith.mulf %9, %9 : tensor<16384xf32, #blocked> loc(#loc11)
    %14 = arith.mulf %12, %12 : tensor<16384xf32, #blocked> loc(#loc12)
    %15 = arith.addf %13, %14 : tensor<16384xf32, #blocked> loc(#loc13)
    %16 = tt.splat %arg2 : !tt.ptr<f32> -> tensor<16384x!tt.ptr<f32>, #blocked> loc(#loc14)
    %17 = tt.addptr %16, %4 : tensor<16384x!tt.ptr<f32>, #blocked>, tensor<16384xi32, #blocked> loc(#loc14)
    tt.store %17, %15, %6 : tensor<16384x!tt.ptr<f32>, #blocked> loc(#loc15)
    tt.return loc(#loc16)
  } loc(#loc)
} loc(#loc)
```

- 答案是不会，将Tile size 增大到16384，执行时间依然稳定
- 翻看 Triton 编译器 IR 不难发现，秘密武器是 Triton 的 layout 系统，能根据数据的访问属性为操作数规划合适的线程映射方案

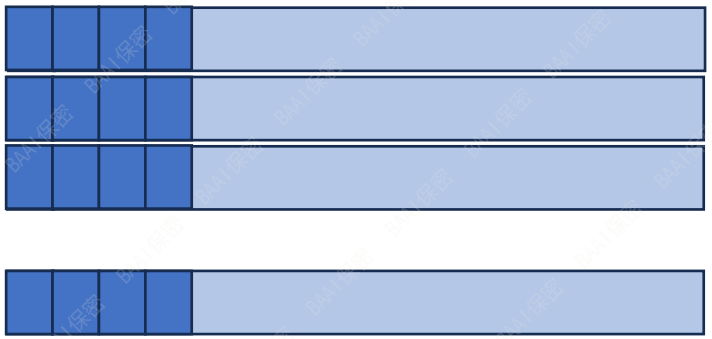
Triton 隐藏了什么


```
add.s64     %rd1, %rd97, %rd100;
add.s64     %rd2, %rd1, 4;
add.s64     %rd3, %rd1, 8;
add.s64     %rd4, %rd1, 12;
add.s64     %rd5, %rd1, 8192;
add.s64     %rd6, %rd1, 8196;
add.s64     %rd7, %rd1, 8200;
add.s64     %rd8, %rd1, 8204;
add.s64     %rd9, %rd1, 16384;
add.s64     %rd10, %rd1, 16388;
add.s64     %rd11, %rd1, 16392;
add.s64     %rd12, %rd1, 16396;
add.s64     %rd13, %rd1, 24576;
add.s64     %rd14, %rd1, 24580;
add.s64     %rd15, %rd1, 24584;
add.s64     %rd16, %rd1, 24588;
add.s64     %rd17, %rd1, 32768;
add.s64     %rd18, %rd1, 32772;
add.s64     %rd19, %rd1, 32776;
add.s64     %rd20, %rd1, 32780;
add.s64     %rd21, %rd1, 40960;
add.s64     %rd22, %rd1, 40964;
add.s64     %rd23, %rd1, 40968;
add.s64     %rd24, %rd1, 40972;
add.s64     %rd25, %rd1, 49152;
add.s64     %rd26, %rd1, 49156;
add.s64     %rd27, %rd1, 49160;
add.s64     %rd28, %rd1, 49164;
add.s64     %rd29, %rd1, 57344;
add.s64     %rd30, %rd1, 57348;
add.s64     %rd31, %rd1, 57352;
add.s64     %rd32, %rd1, 57356;
```

```
#blocked = #ttg.blocked<{
    sizePerThread = [4],
    threadsPerWarp = [32],
    warpsPerCTA = [16],
    order = [0]
}>

tensor<16384xi32, #blocked>
```

block layout

Per thread



每个线程访问的是2d strided tensor

shape = [8, 4]
strides = [8192, 4]

Triton 隐藏了什么

小提示：

- 由于Triton隐藏了循环展开，程序员容易忽视 BLOCK_SIZE 过大造成的代码膨胀

```
2078616 Aug 11 15:27 /home/tongxin/.triton/cache/H6AFDQTHCKTSANA47ZG6PRIJJC7VGD7E7HGDH6HFCSPAG2OPJALQ/elementwise_sumOfSquares.cubin
   1006 Aug 11 15:27 /home/tongxin/.triton/cache/H6AFDQTHCKTSANA47ZG6PRIJJC7VGD7E7HGDH6HFCSPAG2OPJALQ/elementwise_sumOfSquares.json
2245493 Aug 11 15:25 /home/tongxin/.triton/cache/H6AFDQTHCKTSANA47ZG6PRIJJC7VGD7E7HGDH6HFCSPAG2OPJALQ/elementwise_sumOfSquares.llir
2359150 Aug 11 15:25 /home/tongxin/.triton/cache/H6AFDQTHCKTSANA47ZG6PRIJJC7VGD7E7HGDH6HFCSPAG2OPJALQ/elementwise_sumOfSquares.ptx
```

Triton 隐藏了什么

- Triton 系统化了 Layout 概念
  - Triton 的 Layout system 是其核心架构组件
  - 通过特定的Layout控制存储层次间的数据移动，达到优化性能的目的
  - 对比不同编程系统
    - CUDA，代码手动定义
    - CUTE，提供特定API，半系统化定义
    - Triton，系统化，自动定义

Triton 隐藏了什么

- Linear Layout
  - LL 定义为物理地址张量空间到逻辑索引张量空间的索引变换
  - 物理地址：*(warp-id, thread-id, register-id)*
  - 逻辑索引：1-d，2-d，3-d 张量
  - 关键洞察：索引变换是strides的线性组合，如果strides均为2的幂（GPU和AI系统中常见），则各个维度的坐标可以恰好拼成二值向量，张量空间之间的坐标映射关系可以定义成两个二值向量之间的线性变换，而变换的元素操作是二值mod和 xor

[4, 8] -> [16, 2]

[t, r] [i, j]

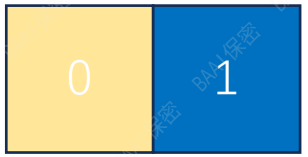
	t		r		
j	1.	0.	0.	0.	0.
	0.	0.	0.	0.	1.
	1.	0.	0.	0.	0.
i	0.	0.	1.	0.	0.
	0.	0.	0.	1.	0.

Triton 隐藏了什么

参考文献：https://arxiv.org/pdf/2505.23819

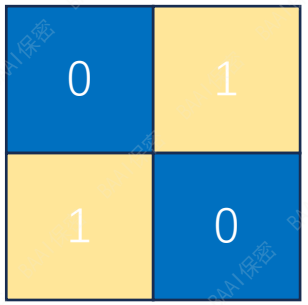
- Linear Layout
  - 换一个角度，LL可以看成 f: {0, 1}^N -> {0, 1}^M，f 在模的意义下保持线性

1 表示 F(x) = 1 * x

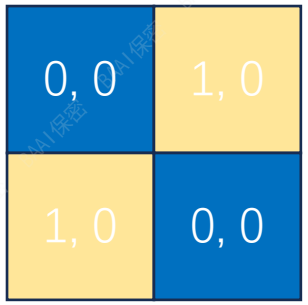


0 表示 F(x) = 0 * x

(1, 1) 表示 F(x, y) = x + y



((1, 1), (0, 0)
表示 F(x, y) = (x + y, 0)



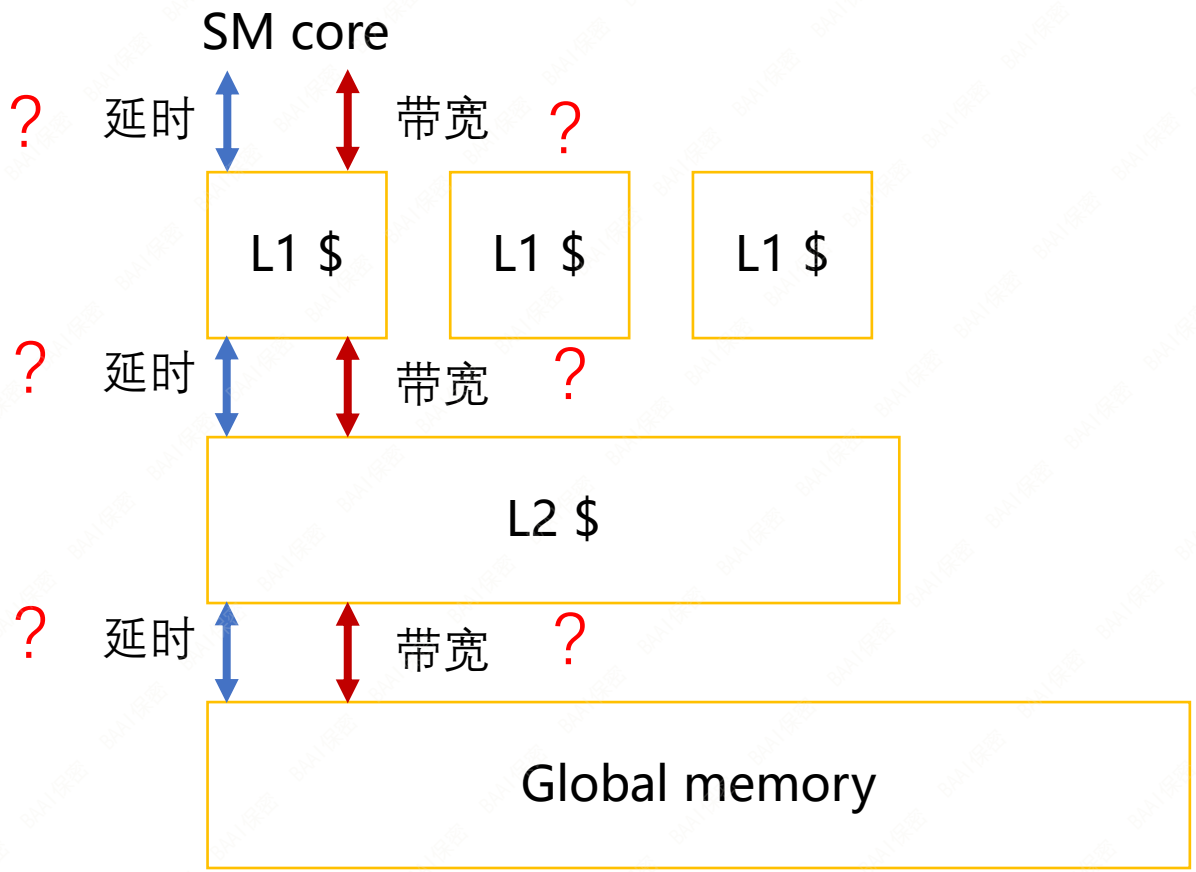
Triton 隐藏了什么

参考文献：https://arxiv.org/pdf/2505.23819

- Linear Layout
  - 更多内容请参考，Linear Layouts: Robust Code Generation of Efficient Tensor Computation Using F2，Keren Zhou et al. https://arxiv.org/pdf/2505.23819
  - Triton 实现请参考 https://github.com/openai/triton

认识访存延时和带宽

- 常识：延时随 L1 -> L2 -> MM 逐层增加，带宽减少
- 问题：GPU架构特点与延时/带宽之间的关系，如何影响kernel编程和优化?



认识访存延时和带宽

- 常识：延时随 L1 -> L2 -> MM 逐层增加，带宽减少
- 问题：GPU架构特点与延时/带宽之间的关系，如何影响kernel编程和优化?

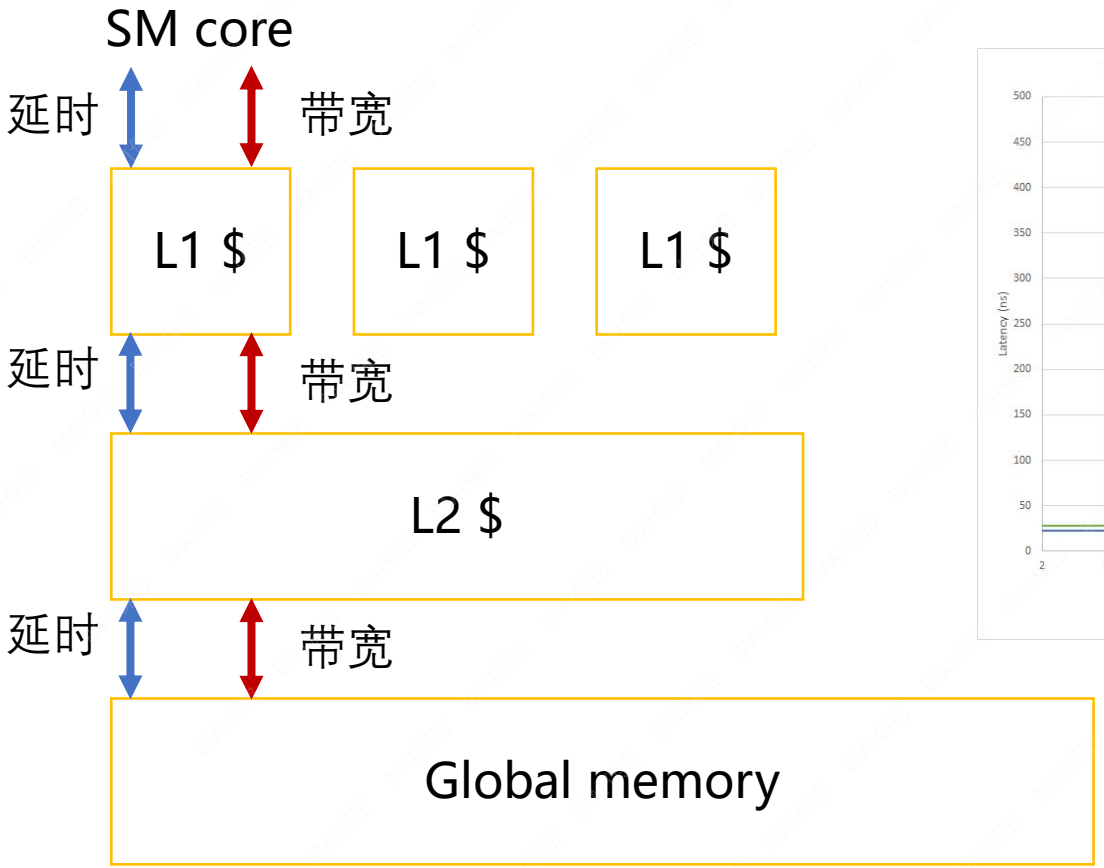
Ampere/Hopper 的L2$采用分区架构，导致近端与远端L2延时不均衡

Figure 6. GA100 Full GPU with 128 SMs (A100 Tensor Core GPU has 108 SMs)

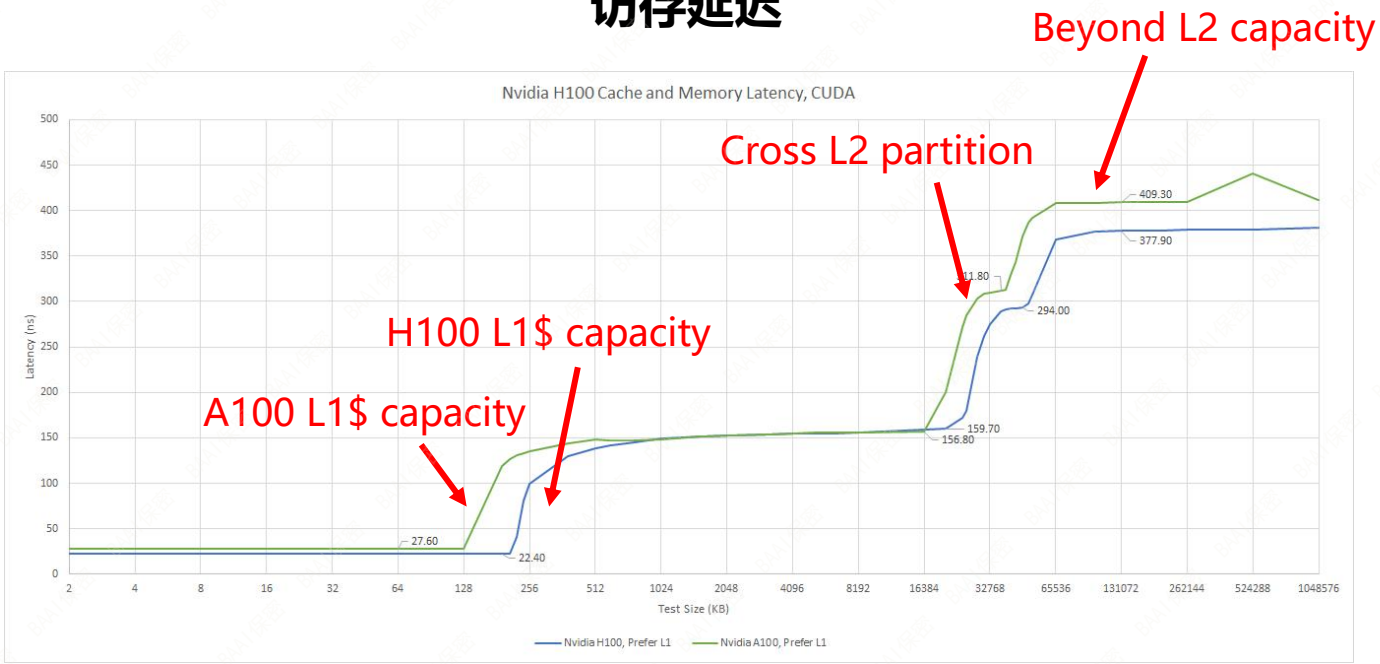
认识访存延时和带宽

- 常识：延时随 L1 -> L2 -> MM 逐层增加，带宽减少
- 问题：GPU架构特点与延时/带宽之间的关系，如何影响kernel编程和优化？

Ampere/Hopper 的L2$采用分区架构，导致近端与远端L2延时不均衡



访存延迟



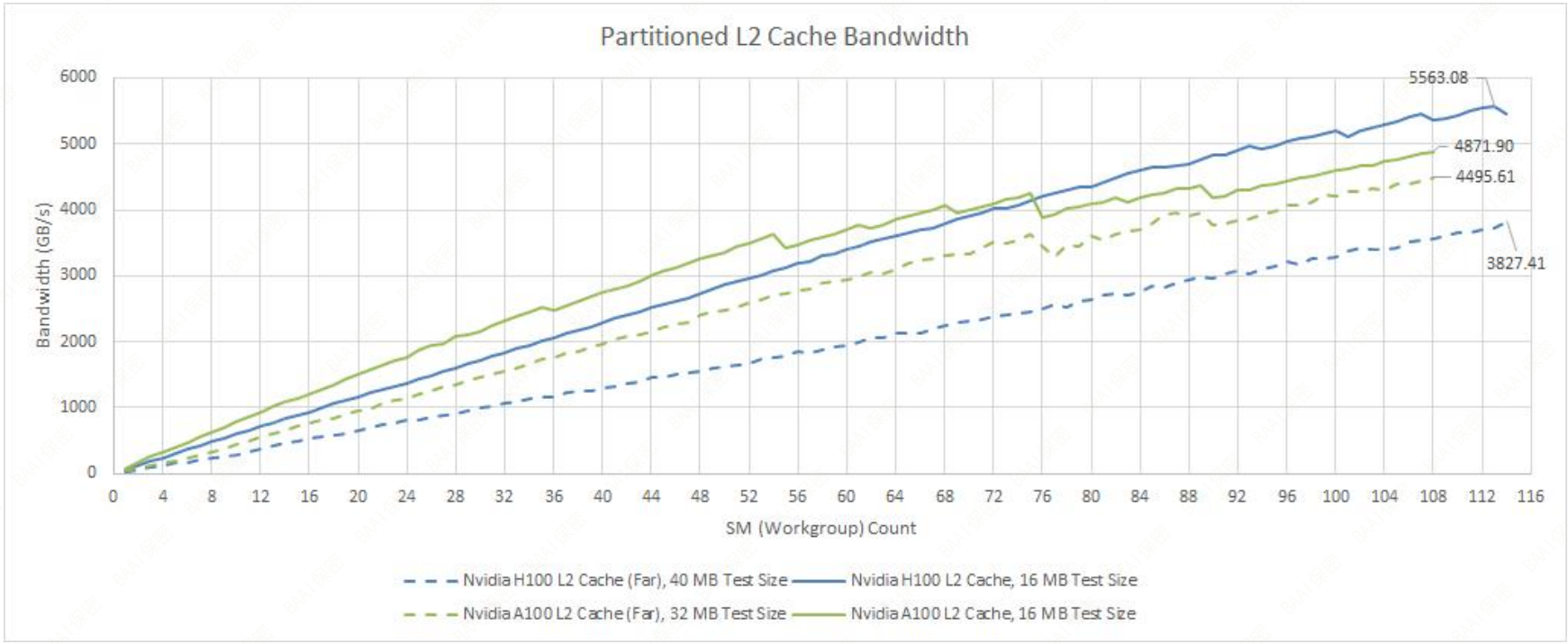
A100，H100 访存延迟参考数据

来源：https://chipsandcheese.com/p/nvidias-h100-funny-l2-and-tons-of-bandwidth

认识访存延时和带宽

- 常识：延时随 L1 -> L2 -> MM 逐层增加，带宽减少
- 问题：GPU架构特点与延时/带宽之间的关系，如何影响kernel编程和优化?

访存带宽



来源：https://chipsandcheese.com/p/nvidias-h100-funny-l2-and-tons-of-bandwidth

认识访存延时和带宽

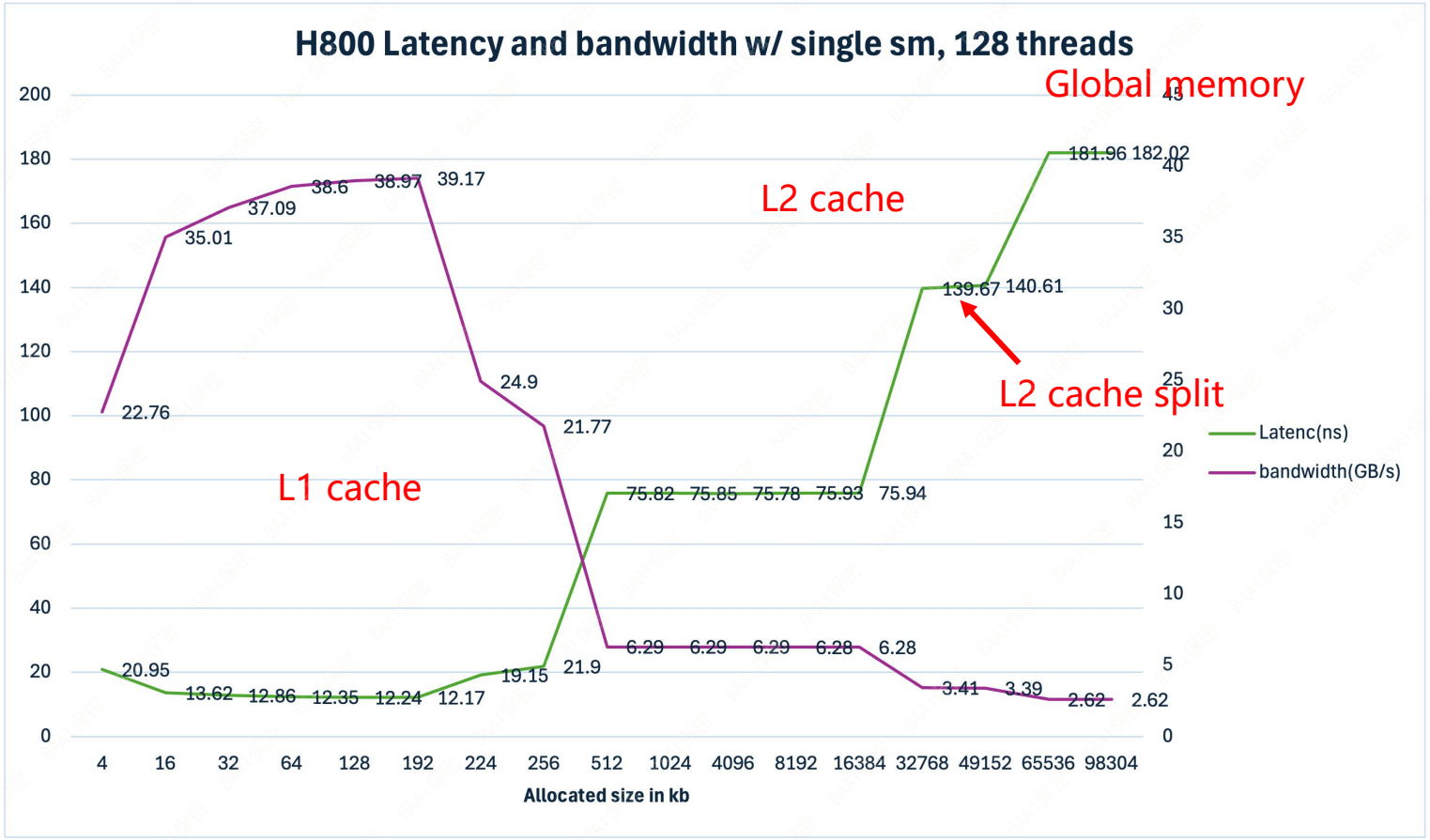
- 常识：延时随 L1 -> L2 -> MM 逐层增加，带宽减少
- 问题：GPU架构特点与延时/带宽之间的关系，如何影响kernel编程和优化?

通过简单CUDA程序复现
延时和带宽测量结果

线程块数量 = 1
线程块大小 = 128

右图是通过调整访存足迹估算H800 GPU在SM无竞争状态下的访存延时和带宽

理论估算无竞争前提下带宽
= #threads * elementsize / 延时



实测延时20.95ns，实测带宽 22.76G/s，估算带宽 24.4G/s
实测延时181.96ns，实测带宽 2.62G/s，估算带宽 2.81G/s

思考 ~7%误差原因?

认识访存延时和带宽

- 常识：延时随 L1 -> L2 -> MM 逐层增加，带宽减少
- 问题：GPU架构特点与延时/带宽之间的关系，如何影响kernel编程和优化？

拐点位置 2：总数据量超过 L2 cache，开始访问 Global memory，由于缺乏并行线程和指令级并行，因此带宽利用率保持恒定且较低的水平

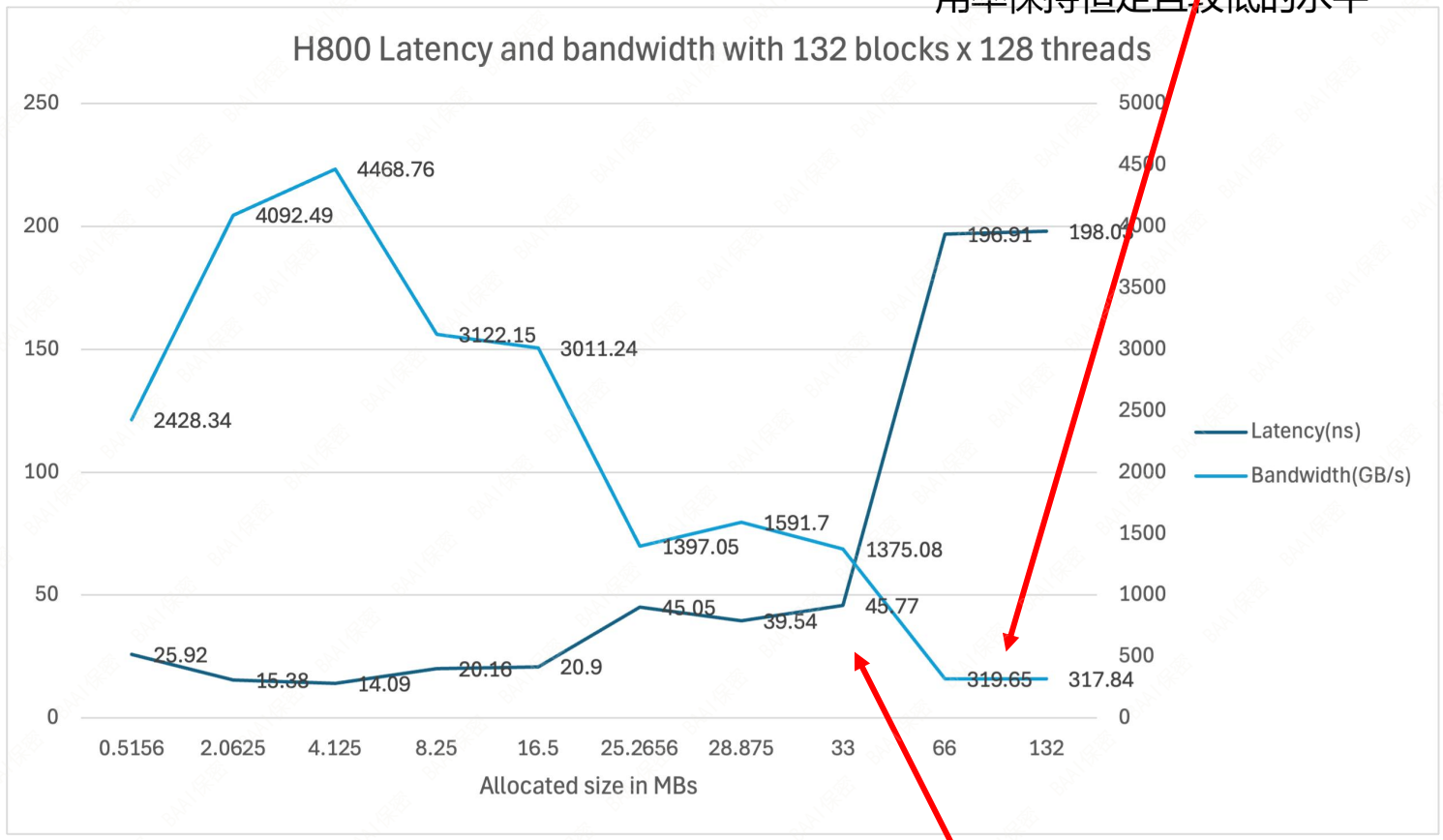
通过简单CUDA程序复现
延时和带宽测量结果

线程块数量 = 132 (SM数量)
线程块大小 = 128

此时不存在TLP及ILP影响

低Occupancy

带宽利用低迷



实测延时25.92ns，实测带宽 2428G/s，估算带宽 2607G/s
实测延时190.51ns，实测带宽 319G/s，估算带宽 354G/s

误差~7%, ~10%

拐点位置 1：总数据量分配超过34MB时，线程块足迹超过L1 cache 大小，L2 cache 主导访存流量

认识访存延时和带宽

- 常识：延时随 L1 -> L2 -> MM 逐层增加，带宽减少
- 问题：GPU架构特点与延时/带宽之间的关系，如何影响kernel编程和优化?

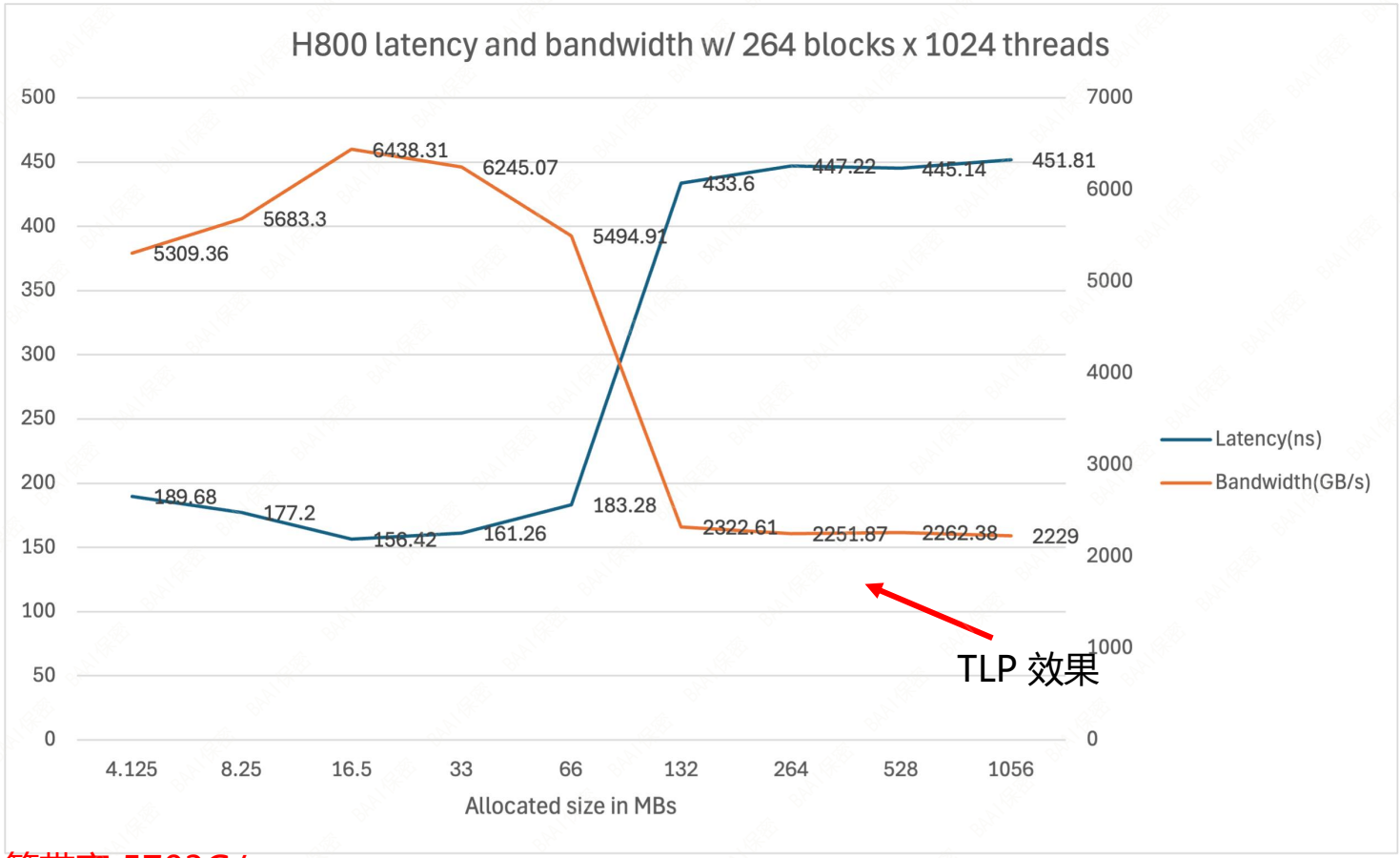
通过简单CUDA程序复现
延时和带宽测量结果

线程块数量 = 264
线程块大小 = 1024

Occupancy 打满

通过TLP调高带宽利用

~70% SOL



实测延时189.6ns，实测带宽 5309G/s，估算带宽 5703G/s
实测延时445.14ns，实测带宽 2251G/s，估算带宽 2429G/s

误差~15%, ~8%

认识访存延时和带宽

- 常识：延时随 L1 -> L2 -> MM 逐层增加，带宽减少
- 问题：GPU架构特点与延时/带宽之间的关系，如何影响kernel编程和优化?

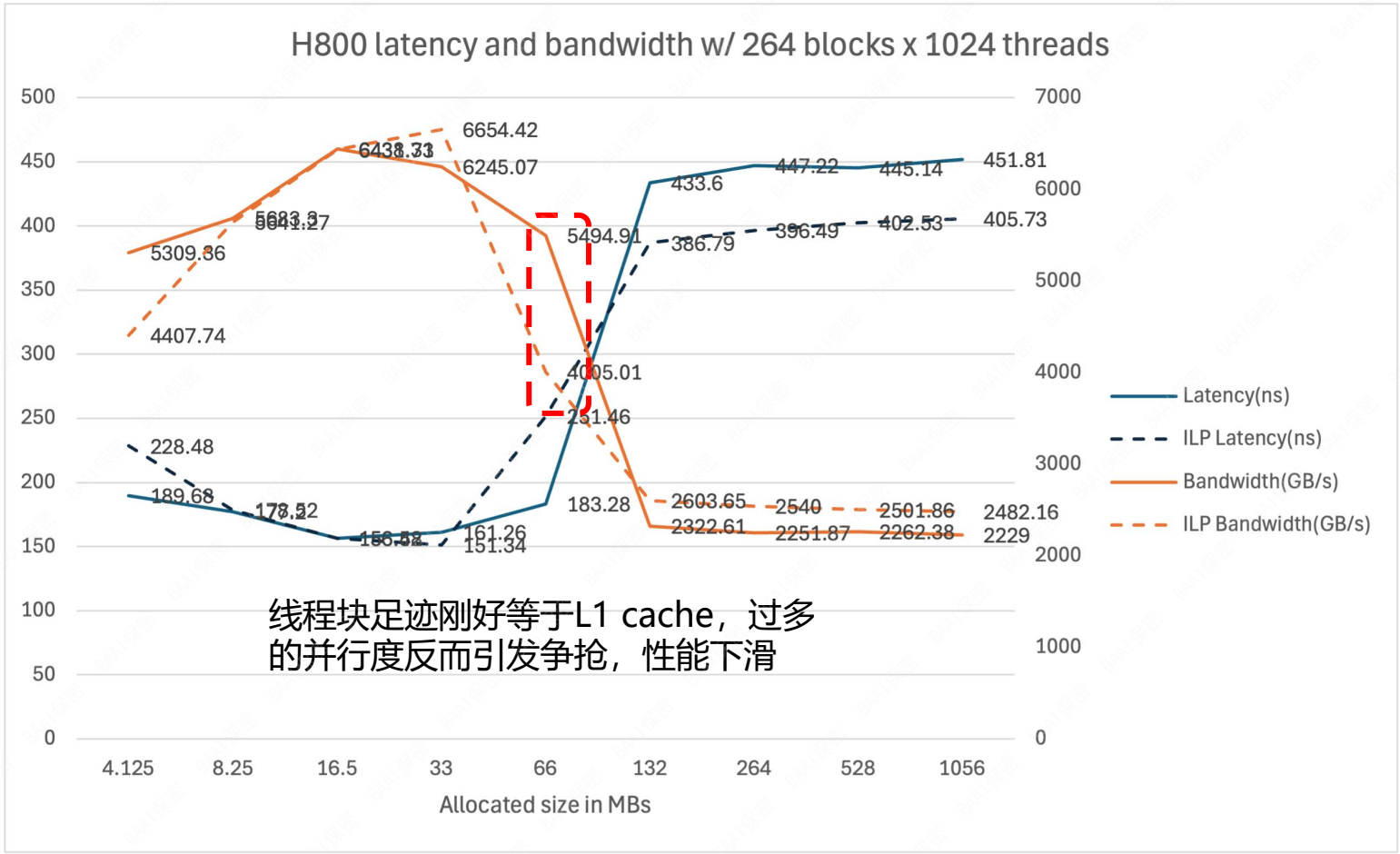
通过简单CUDA程序复现
延时和带宽测量结果

线程块数量 = 264
线程块大小 = 1024

Occupancy 打满

在TLP基础上通过简单的手动unroll提升ILP

达到 ~75% SOL



认识访存延时和带宽

- 常识：延时随 L1 -> L2 -> MM 逐层增加，带宽减少
- 问题：GPU架构特点与延时/带宽之间的关系，如何影响kernel编程和优化?

是否巧合？一个简单的纸巾算数题

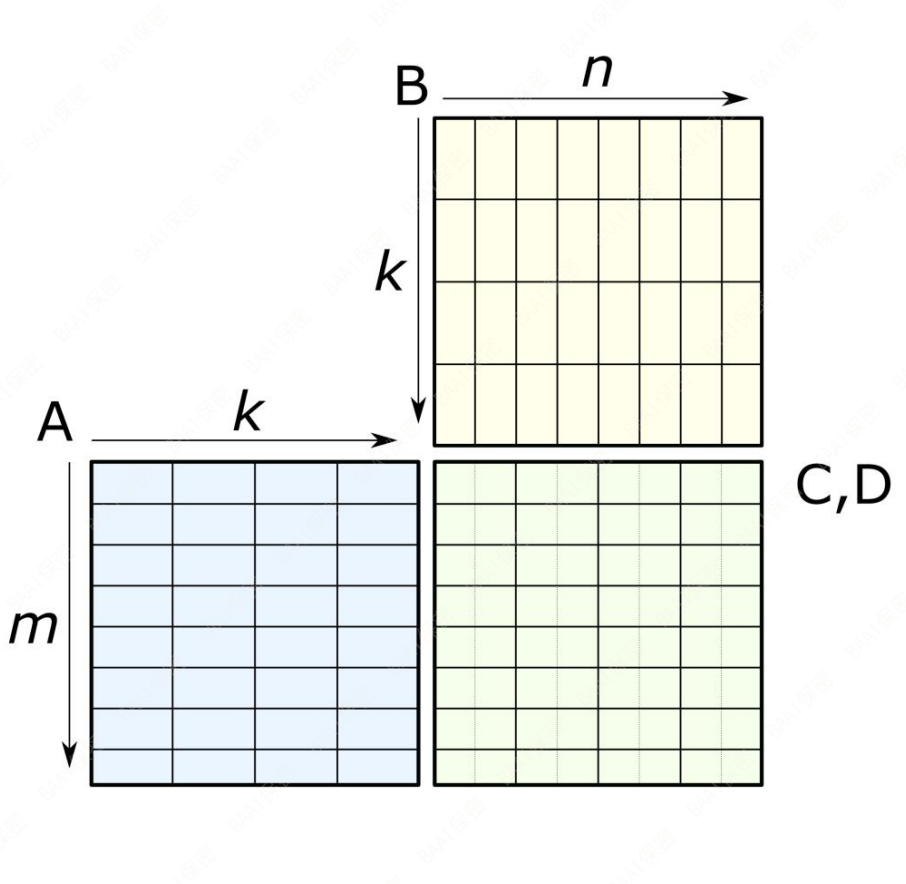
- 假设GPU充分并行时的延时为 400ns
- 264个线程块单次同时读写连续4KB数据，恰好达到 4*264KB/400ns = 2.64 TB/s 有效带宽，比较接近SOL
- 4KB 也是典型半精度模型推理中单头 block page大小 (block_size * head_dim * 2 = 16 * 128 * 2)

认识访存延时和带宽

- 常识：延时随 L1 -> L2 -> MM 逐层增加，带宽减少
- 问题：GPU架构特点与延时/带宽之间的关系，如何影响kernel编程和优化?
  - GPU在满足大量SM并行执行的同时，竭尽其力达到带宽和延时的平衡
    - 精细分化存储层次和容量配比，甚至L2分区以提高效率
    - 通过SM内复杂设计隐藏高延时，支撑高带宽
      - TLP：提升动态warp数量，warp动态调度
  - 软件层面，通过优化kernel代码减少访存阻塞，提升实际有效带宽 （内容略过）
    - ILP：合并、unroll 增加 bytes in flight
    - TLP：提升 warp occupancy
    - 优化L1/shared memory 使用
    - 异步数据拷贝，消除访存阻塞带来的性能损耗

Tensor core 编程

- Tensor core 操作抽象
  - 执行的计算：D = A @ B + C
  - 形状表示：m x n x k
  - 指令：mma, wgmma

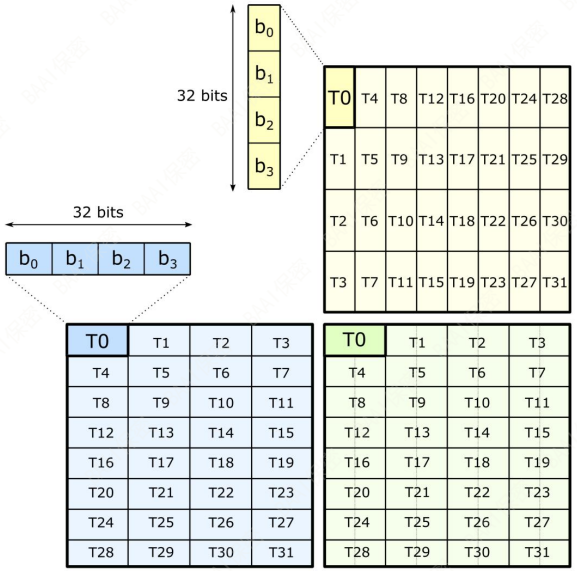


https://developer.download.nvidia.cn/video/gputechconf/gtc/2020/presentations/s21745-developing-cuda-kernels-to-push-tensor-cores-to-the-absolute-limit-on-nvidia-a100.pdf

Tensor core 编程

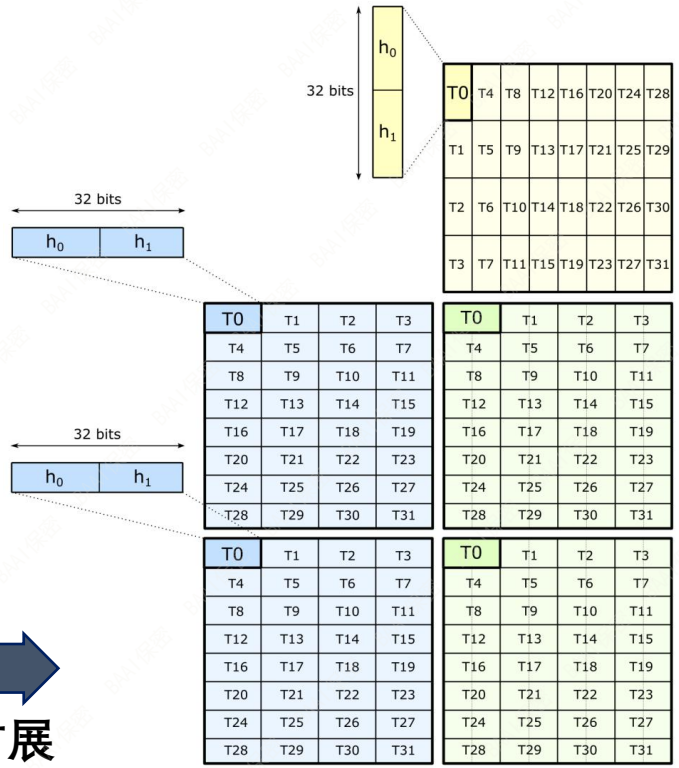
- 编程
  - 线程集合操作
  - 32个线程平摊 A, B, C, D 操作数

8-by-8-by-16



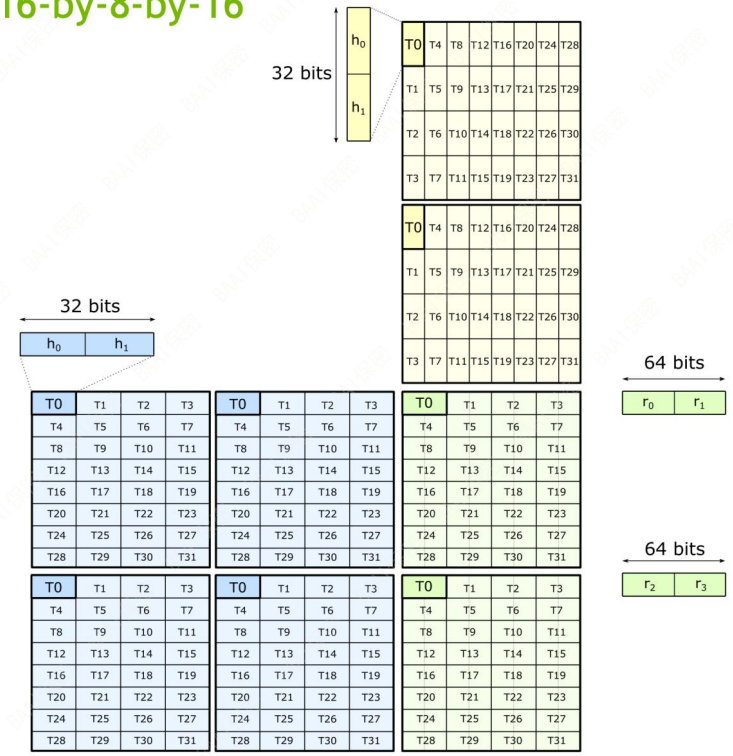
8 8 32

16-by-8-by-8



M维扩展

16-by-8-by-16



K维度扩展

16 16 32

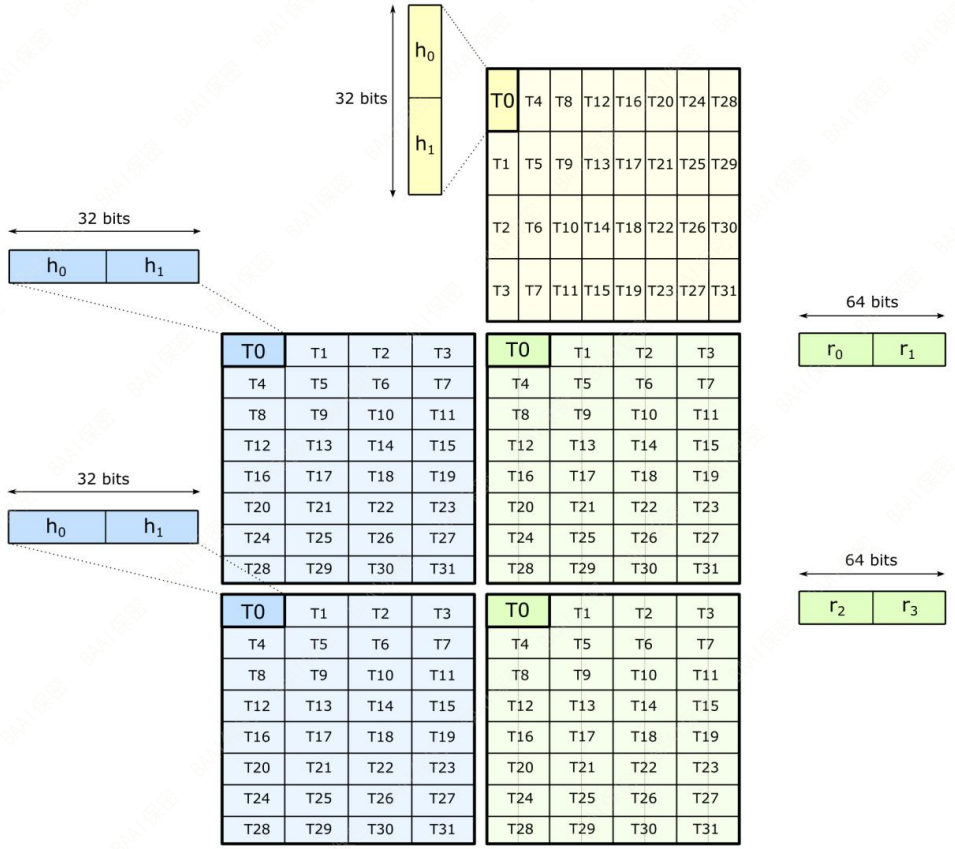
https://developer.download.nvidia.cn/video/gputechconf/gtc/2020/presentations/s21745-developing-cuda-kernels-to-push-tensor-cores-to-the-absolute-limit-on-nvidia-a100.pdf

Tensor core 编程

mma.sync.aligned
(via inline PTX)

```
float          D[4];
uint32_t const A[2];
uint32_t const B;
float    const C[4];

// Example targets 16-by-8-by-8 Tensor Core operation
asm(
  "mma.sync.aligned.m16n8k8.row.col.f32.f16.f16.f32 "
  "  { %0, %1, %2, %3 }, "
  "  { %4, %5},          "
  "    %6,               "
  "  { %7, %8, %9, %10 };"
  :
    "=f"(D[0]), "=f"(D[1]), "=f"(D[2]), "=f"(D[3])
  :
    "r"(A[0]), "r"(A[1]),
    "r"(B),
    "f"(C[0]), "f"(C[1])
);
```

16-by-8-by-8

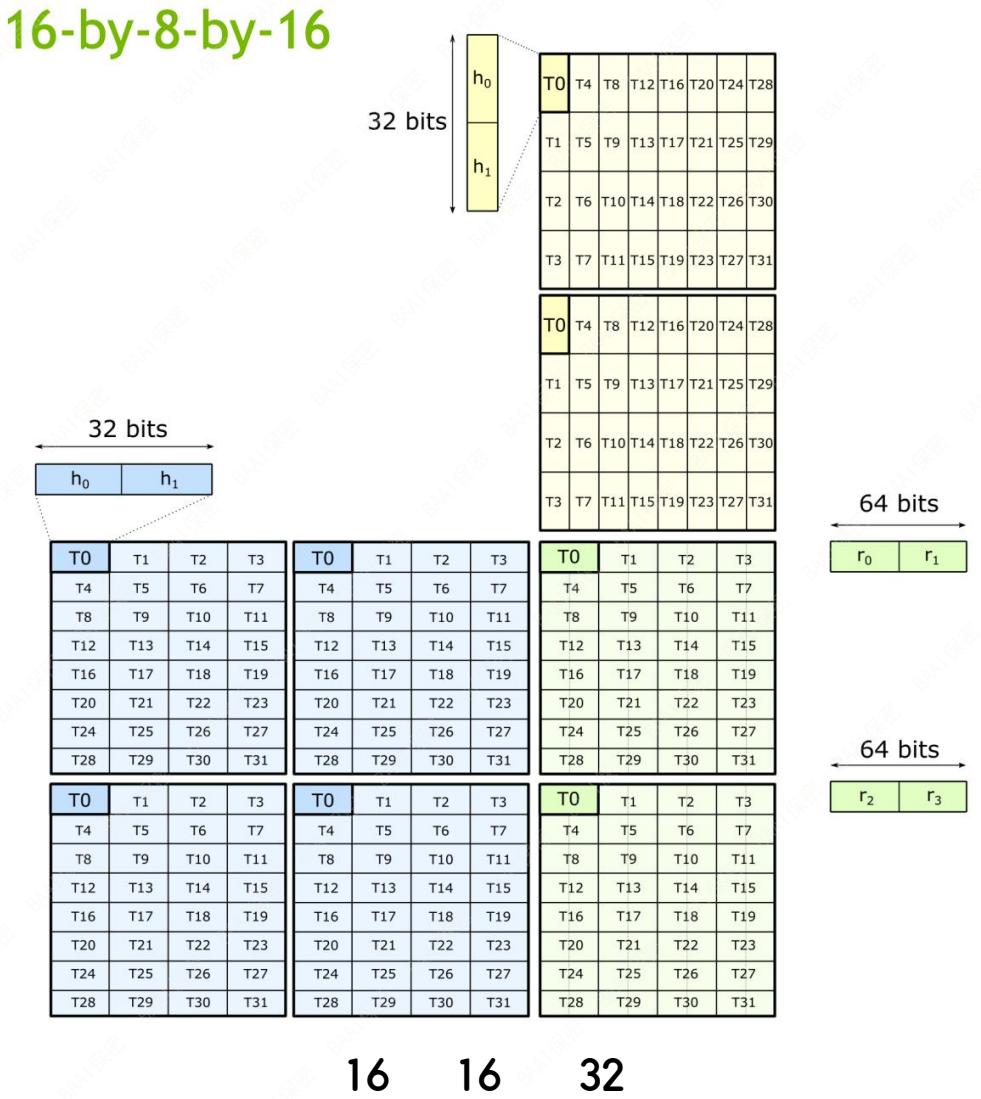


https://developer.download.nvidia.cn/video/gputechconf/gtc/2020/presentations/s21745-developing-cuda-kernels-to-push-tensor-cores-to-the-absolute-limit-on-nvidia-a100.pdf

Tensor core 编程

mma.sync.aligned
(via inline PTX)

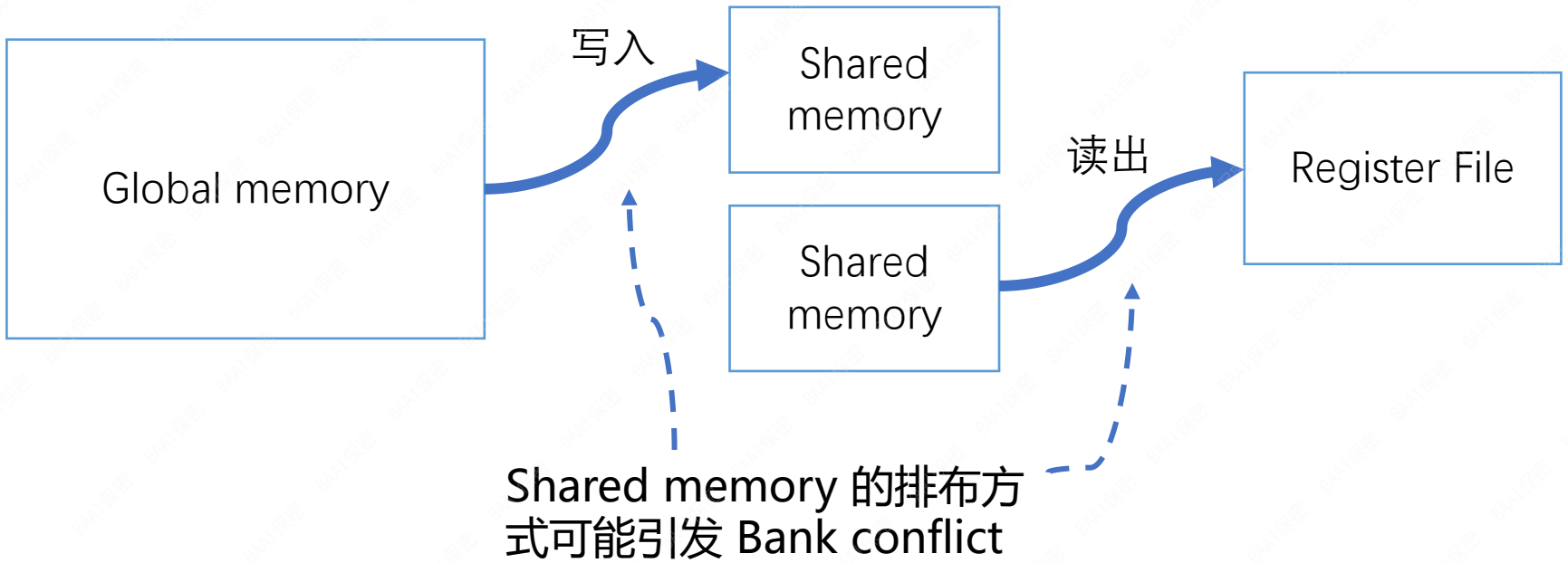
```
float          D[4];
uint32_t const A[4];
uint32_t const B[2];
float    const C[4];

// Example targets 16-by-8-by-32 Tensor Core operation
asm(
  "mma.sync.aligned.m16n8k16.row.col.f32.f16.f16.f32 "
  "   { %0, %1, %2, %3 },     "
  "   { %4, %5, %6, %7 },     "
  "   { %8, %9 },             "
  "   { %10, %11, %12, %13 };"
  :
    "=f"(D[0]), "=f"(D[1]), "=f"(D[2]), "=f"(D[3])
  :
    "r"(A[0]), "r"(A[1]), "r"(A[2]), "r"(A[3]),
    "r"(B[0]), "r"(B[1]),
    "f"(C[0]), "f"(C[1]), "f"(C[2]), "f"(C[3])
);
```



https://developer.download.nvidia.cn/video/gputechconf/gtc/2020/presentations/s21745-developing-cuda-kernels-to-push-tensor-cores-to-the-absolute-limit-on-nvidia-a100.pdf

Tensor core 编程

- 数据移动是tensorcore编程的难点
  - 目标问题：如何将操作数尽可能快的从global memory搬运到满足tensorcore指令布局的寄存器
  - 方法
    - 减少数据移动和寄存器使用：从 global memory 到shared memory 使用异步copy
    - 避免shared memory bank conflicts



Tensor core 编程

- Shared memory bank conflicts
  - 什么是memory bank?
    - Shared memory分成了若干个（通常为32个）大小相同的module，称为banks，多个线程可同时访问落在不同bank上的内存地址
  - 地址映射规则
    - 连续的32bit 内存块地址映射到连续的bank上
  - Bank conflict 发生条件
    - 多个线程读写shared memory时，地址不同且落到同一bank上
    - 广播不引起bank conflict，即多个线程访问同一地址

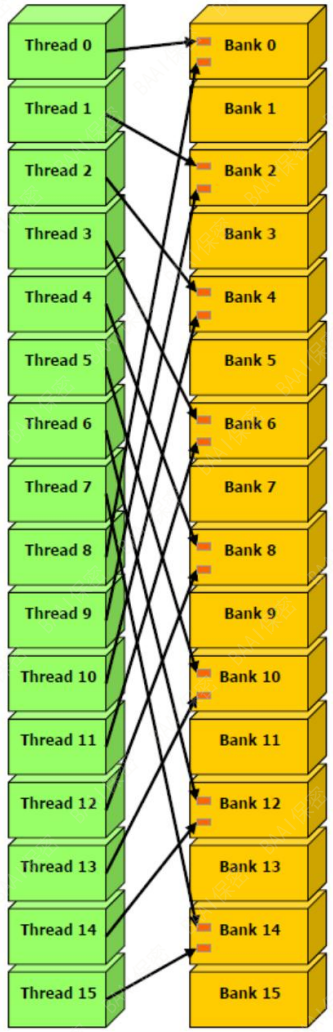
```
__        __                    256   //

//                                32-
                      / 4  % 32  //      32
```

Tensor core 编程

- Bank conflicts 模式
  - Strided access 模式

2-way bank conflicts

```
__          __                              32

//          64-
```



Tensor core 编程

- Bank conflicts 模式
  - 2d 矩阵转置访问模式

```
__global__ void matMult(const float* a, size_t lda,
    const float* b, size_t ldb, float* c, size_t ldc, int n) {
    __shared__ float matA[BLOCK_SIZE][BLOCK_SIZE];
    __shared__ float matB[BLOCK_SIZE][BLOCK_SIZE];
    const int tidc = threadIdx.x;      // c列
    const int tidr = threadIdx.y;      // r行
    const int bidc = blockIdx.x * BLOCK_SIZE;
    const int bidr = blockIdx.y * BLOCK_SIZE;
    ...
    for(j = 0; j < n; j += BLOCK_SIZE) {
        matA[tidr][tidc] = a[(tidr+bidr)*lda+tidc+j];
        matB[tidr][tidc] = b[(tidr+j)*ldb+tidc+bidc];
        __syncthreads();
        for(i = 0; i < BLOCK_SIZE; i++)
            result += matA[tidr][i] * matB[i][tidc];
            ...
```

输入copy 到 shared memory

A 的 r 行与 B 的 c 列乘加

matA [**tidr** * BLOCKSIZE + i]　　matB [i * BLOCKSIZE + **tidc**]

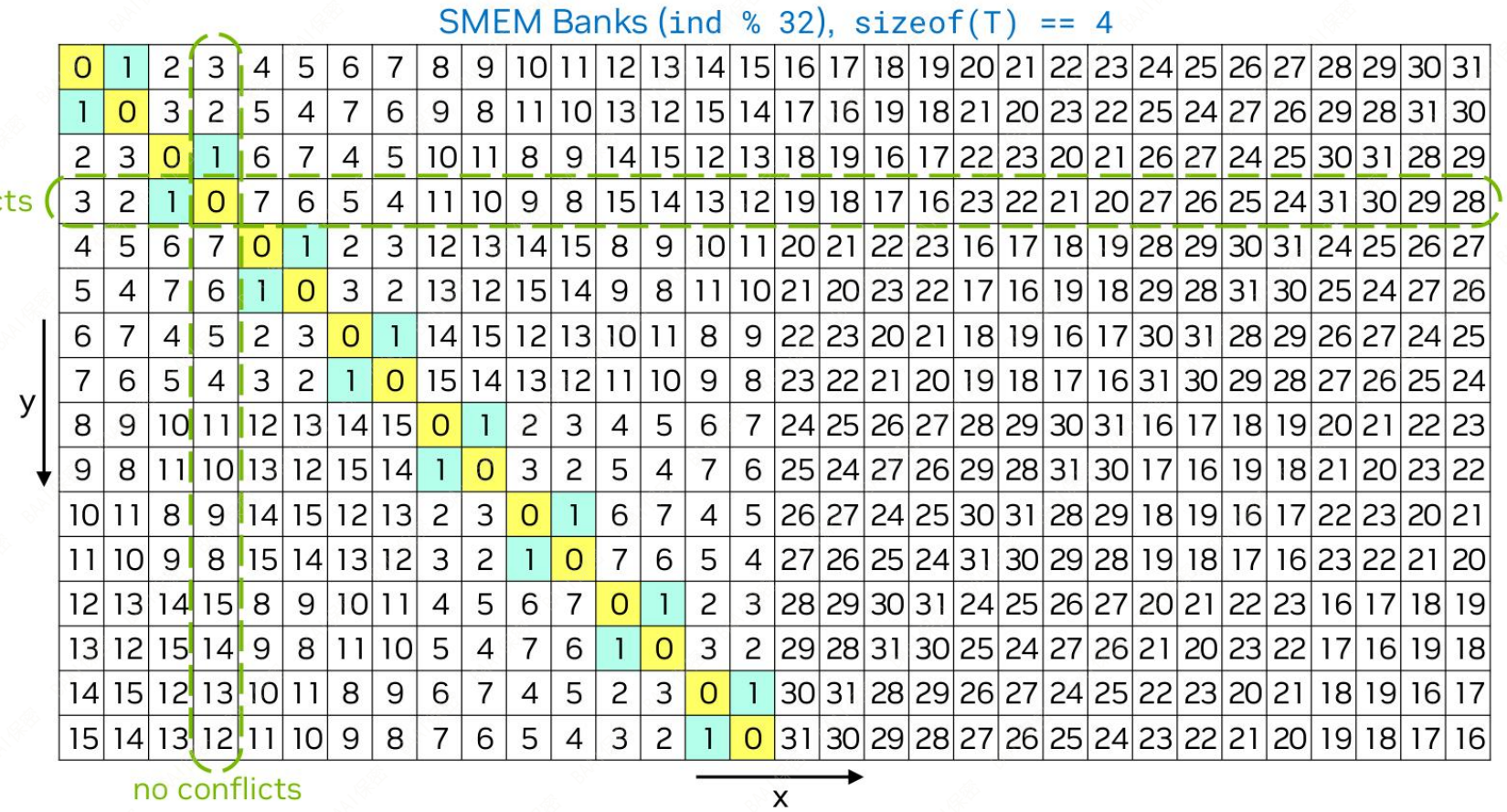
Tensor core 编程

- 如何解决bank conflicts
  - padding
  - **index swizzling**

Index swizzling 的想法类似拉丁方：

1	2	3	4	5
5	1	2	3	4
4	5	1	2	3
3	4	5	1	2
2	3	4	5	1

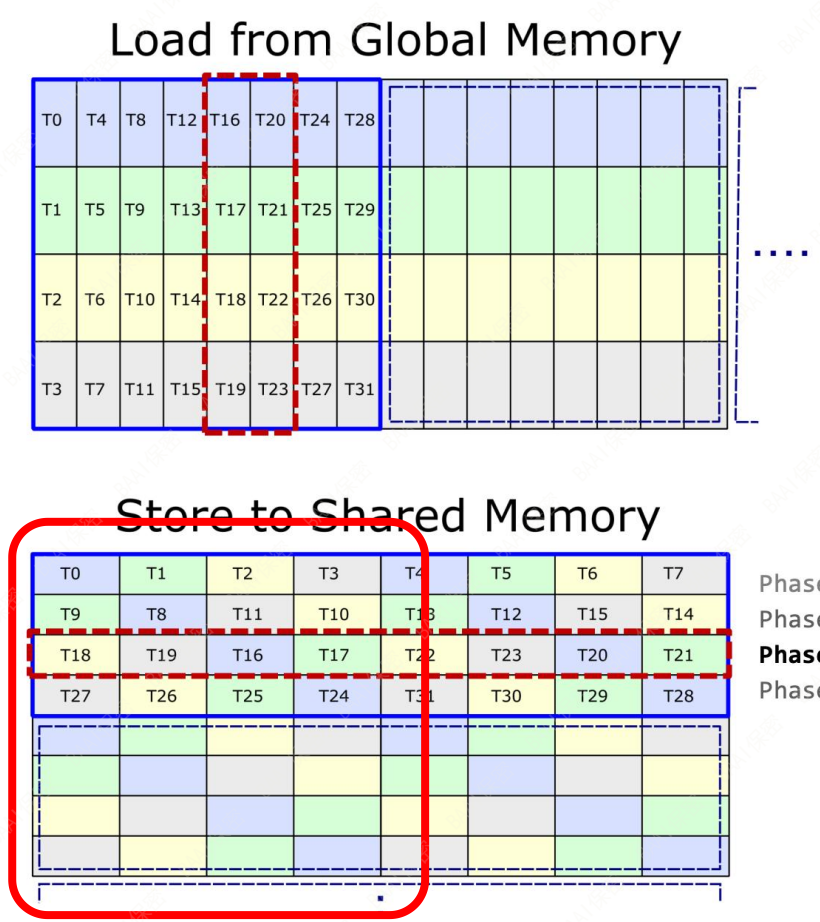
数字对应原地址偏移，例如第3行的0表示从地址偏移0拷贝到地址偏移3。
图中黄色表示原第0列swizzle之后的位置，当32个线程并行读取第0和第1列时仅发生2-way conflicts。如果swizzle 32x32矩阵，则将完全避免conflicts



Source: https://leimao.github.io/blog/CUDA-Shared-Memory-Swizzling/

Tensor core 编程

回到tensorcore的bank conflict问题

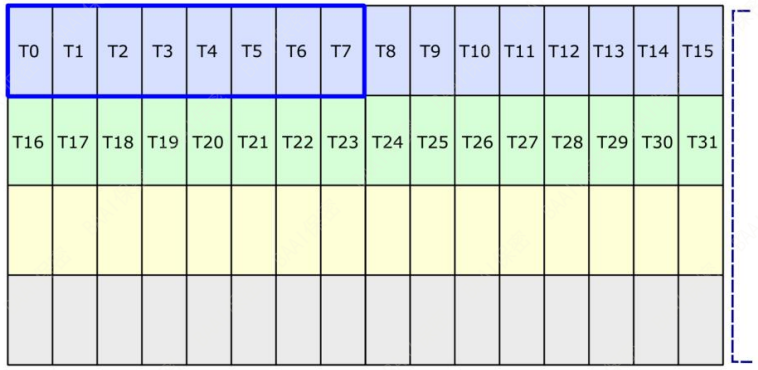


若保持相同layout，则 G -> S, S -> RF
二者之一必然引起bank conflicts

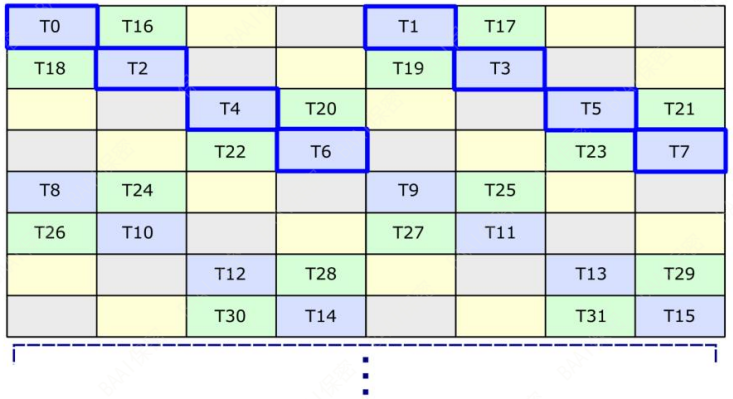
Swizzle layout 避免 bank conflicts

Tensor core 编程

Logical view of threadblock tile



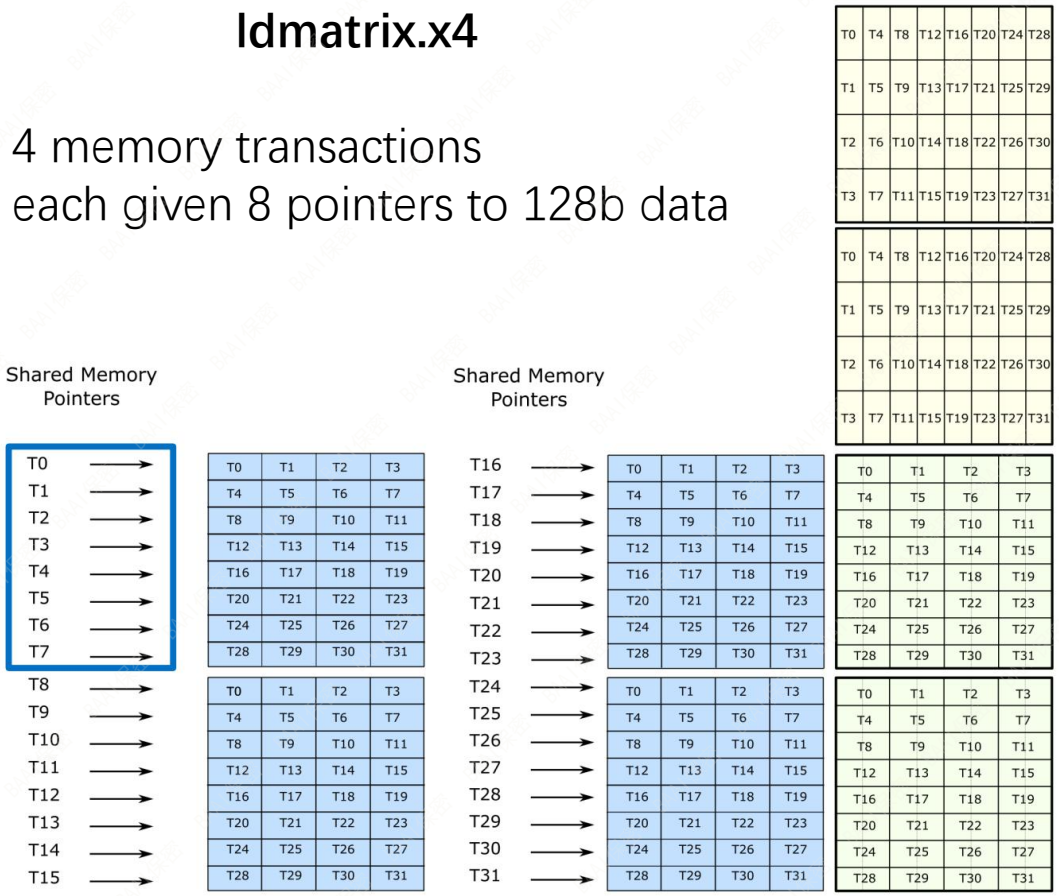
Load Matrix from Shared Memory



每小格 128b = 16 bytes

ldmatrix.x4

4 memory transactions
each given 8 pointers to 128b data



Tensor core 编程

T0	T16			T1	T17		
T18	T2			T19	T3		
		T4	T20			T5	T21
		T22	T6			T23	T7
T8	T24			T9	T25		
T26	T10			T27	T11		
		T12	T28			T13	T29
		T30	T14			T31	T15

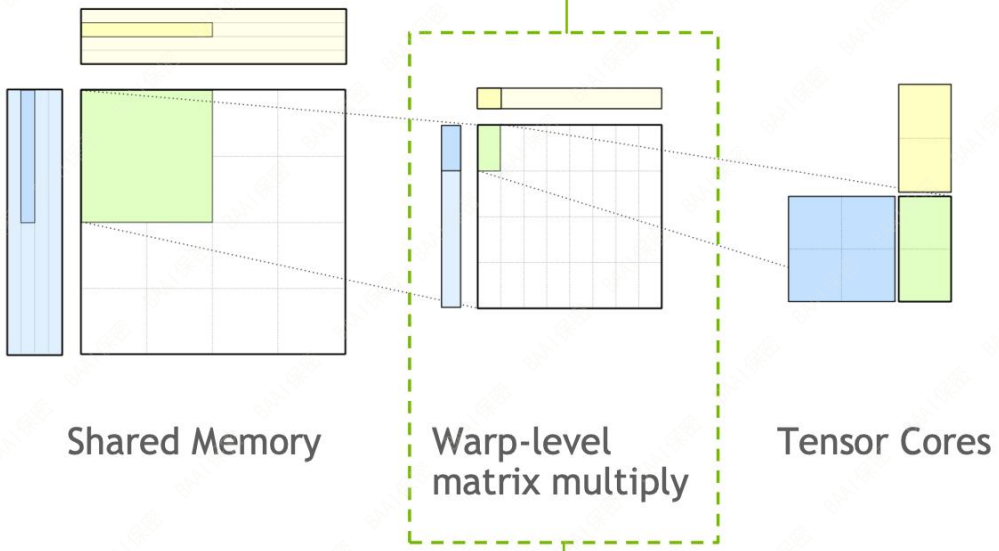
smem_ptr = row_idx * 8 + column_idx;

		T0	T16			T1	T17
		T18	T2			T19	T3
T4	T20			T5	T21		
T22	T6			T23	T7		
		T8	T24			T9	T25
		T26	T10			T27	T11
T12	T28			T13	T29		
T30	T14			T31	T15		

smem_ptr = smem_ptr ^ 2;

Tensor core 编程

Cutlass 代码示例




```
using Mma = cutlass::gemm::warp::DefaultMmaTensorOp<
  GemmShape<64, 64, 16>,
  half_t, LayoutA,                  // GEMM A operand
  half_t, LayoutB,                  // GEMM B operand
  float, RowMajor                   // GEMM C operand
>;

__shared__ ElementA smem_buffer_A[Mma::Shape::kM * GemmK];
__shared__ ElementB smem_buffer_B[Mma::Shape::kN * GemmK];

// Construct iterators into SMEM tiles
Mma::IteratorA iter_A({smem_buffer_A, lda}, thread_id);
Mma::IteratorB iter_B({smem_buffer_B, ldb}, thread_id);

Mma::FragmentA frag_A;
Mma::FragmentB frag_B;
Mma::FragmentC accum;

Mma mma;

accum.clear();

#pragma unroll 1
for (int k = 0; k < GemmK; k += Mma::Shape::kK) {


  iter_A.load(frag_A);  // Load fragments from A and B matrices
  iter_B.load(frag_B);

  ++iter_A; ++iter_B;   // Advance along GEMM K to next tile in A
                        //   and B matrices

                        // Compute matrix product
  mma(accum, frag_A, frag_B, accum);
}
```

迭代器将logical block与swizzle变换后的shared memory映射关系隐藏起来

https://developer.download.nvidia.cn/video/gputechconf/gtc/2020/presentations/s21745-developing-cuda-kernels-to-push-tensor-cores-to-the-absolute-limit-on-nvidia-a100.pdf

总结

- 本次介绍内容回顾
  - GPU并行架构与SIMT线程执行模型
  - 工程角度初步对比CUDA与Triton两种编程模式
  - 初步认识访存时延与带宽
  - Tensorcore相关的基本编程技术
- 其他重要的GPU编程技术与编程问题
  - 不同类型算法的典型实现方案，如Reduction、Gather/Scatter、Gemm、Attn等
  - 如何通过共享内存的异步copy实现线程块级别的软件流水
  - 利用Warp specialization进行producer-consumer风格异步编程
  - 多机通信并行编程模型